CentraleSupélec

université PARIS-SACLAY

Big Data Management & Analytics Master

Partial Differential Equation Solver using Neural Field Turing Machines (NFTM)

Samuel CHAPUIS
samuel.chapuis@student-cs.fr

Lucia Victoria FERNANDEZ SANCHEZ
lucia-victoria.fernandez@student-cs.fr

Alexandra PERRUCHOT-TRIBOULET RODRIGUEZ
alexandra.perruchot-triboulet-rodriguez@student-cs.fr

Github Link

Advisor: Nacéra SEGHOUANI - Nacera.Seghouani@centralesupelec.fr
Advisor: Akash MALHOTRA - akash.malhotra@centralesupelec.fr

Contents

Chapter 1

Introduction

1.1 Context and motivation

1.1.1 Project goals

Our objective is to build a generic neural architecture that can internalize the governing rules of physical systems and then advance their fields over time. The model is trained on challenging PDEs with a focus on fluid mechanics (Navier–Stokes) to stress-test its ability to learn complex, multi-scale, nonlinear dynamics. Beyond matching the training horizon, the same architecture should simulate and extrapolate trajectories past the seen time window, remaining stable and accurate while generalizing across PDE families. Comparing these predictions to reference simulations allows us to identify which ingredients are essential for robustness and transfer.

1.1.2 Minimal mathematical frame

Many PDEs of interest can be written in a (semi-)invariant form:

$$\partial_t u(x,t) = \mathcal{F}\big(u(x,t), \nabla u(x,t), \nabla^2 u(x,t); \theta\big), \quad x \in \Omega,\ t > 0, \tag{1.1}$$

with initial/boundary conditions. Here u is a scalar or vector field; $\mathcal{F}$ encodes advection, diffusion, reaction, constraints; and θ collects physical parameters (viscosity ν, conductivity α, permeability $a(x)$, etc.). Our network seeks to learn a local time-advance operator that approximates $\mathcal{F}$ across multiple PDE families.

1.1.3 Historical milestones: AI and PDEs

- **1990s–2000s**: early universal approximators for simple PDEs; occasional RBF/MLP surrogates.
- **2017–2019**: *Physics-Informed Neural Networks* (PINNs) [13] enforce PDE residuals in the loss; good on simple geometries, sensitive to stiffness and turbulent regimes.
- **2020**: *Fourier Neural Operator* (FNO) [7] and *Neural Operators* [5] learn the operator between function spaces; reference benchmarks on Burgers, incompressible Navier–Stokes, Darcy.
- **2021–2024**: rise of spatio-temporal architectures (Transformers, ConvNeXt) for continuous fields; work on numerical stability, spectral regularization [12], and conservation of physical quantities.

- **NFTM (2025)**: Neural Field Turing Machine (Malhotra & Seghouani) [10] combines **continuous spatial memory + read/write heads + a neural controller** (CNN/RNN/Transformer). It resembles an explicit scheme: local read, compute an update, local write. Our work builds on this idea to generalize across PDEs.

1.2 Fluid Mechanics Overview

1.2.1 Why Navier–Stokes?

Navier–Stokes models is a canonical set of nonlinear PDEs governing fluid flow, encompassing a wide range of phenomena from laminar to turbulent regimes. Their complexity and multi-scale nature make them an ideal testbed for evaluating the capabilities of neural architectures in learning physical dynamics. Successfully modeling Navier–Stokes would demonstrate the potential of AI-driven approaches in computational fluid dynamics (CFD) and beyond.

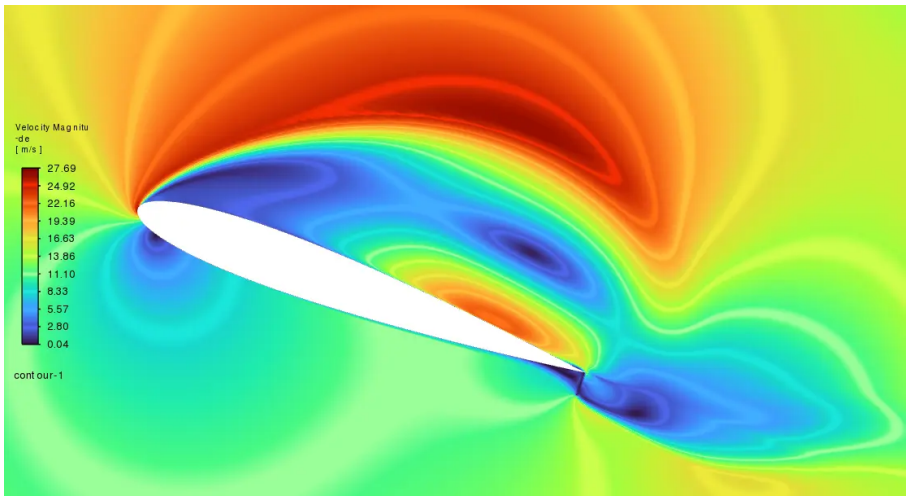


Figure 1.1: Computational fluid dynamics simulation of flow over an airfoil. Accurately resolving the velocity and pressure fields requires solving the Navier–Stokes equations at high resolution, which is computationally expensive. Neural surrogates aim to replicate such simulations at a fraction of the cost.

1.2.2 Governing equations

The Navier–Stokes equations describe the motion of fluid substances and are derived from fundamental conservation laws: mass, momentum, and energy. They can be expressed as follows:

$$\text{Mass Conervation}: \quad \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \tag{1.2}$$

$$\text{Momentum Conervation}: \quad \frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u}) = -\nabla p + \nabla \cdot \boldsymbol{\Sigma} + \rho \mathbf{g} \tag{1.3}$$

$$\text{Energy Conervation}: \quad \frac{\partial E}{\partial t} + \nabla \cdot ((E + p)\mathbf{u}) = \nabla \cdot (\boldsymbol{\tau} \cdot \mathbf{u}) - \nabla \cdot \mathbf{q} + \rho \mathbf{u} \cdot \mathbf{g} \tag{1.4}$$

Where all the variables are defined as:

- ρ: fluid density [$\mathrm{kg \cdot m^{-3}}$]
- $\mathbf{u} = (u, v, w)$: velocity field [$\mathrm{m \cdot s^{-1}}$]
- $\nabla \cdot (\rho \mathbf{u})$: divergence of mass flux
- $\partial \rho / \partial t$: local time rate of change of density
- $\rho \mathbf{u}$: momentum density [$\mathrm{kg \cdot m^{-2} \cdot s^{-1}}$]
- $\nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u})$: convective momentum transport
- $-\nabla p$: pressure gradient force per unit volume
- $\boldsymbol{\Sigma}$: viscous stress tensor [Pa]
- $\rho \mathbf{g}$: body force density (e.g. gravity) [$\mathrm{N \cdot m^{-3}}$]
- $E = \rho\big(e + \frac{1}{2}|\mathbf{u}|^2\big)$: total energy density (internal + kinetic)
- p: pressure [Pa]
- $\boldsymbol{\tau}$: stress tensor (viscous + pressure)
- $\mathbf{q}$: heat flux vector [$\mathrm{W \cdot m^{-2}}$]
- $\rho \mathbf{u} \cdot \mathbf{g}$: work done by body forces

These equations require a viscosity model (e.g., Newtonian fluid) and an equation of state (e.g., ideal gas law or van der Waals equation) to close the system. They form the foundation for simulating fluid dynamics in various applications, from aerodynamics to weather forecasting.

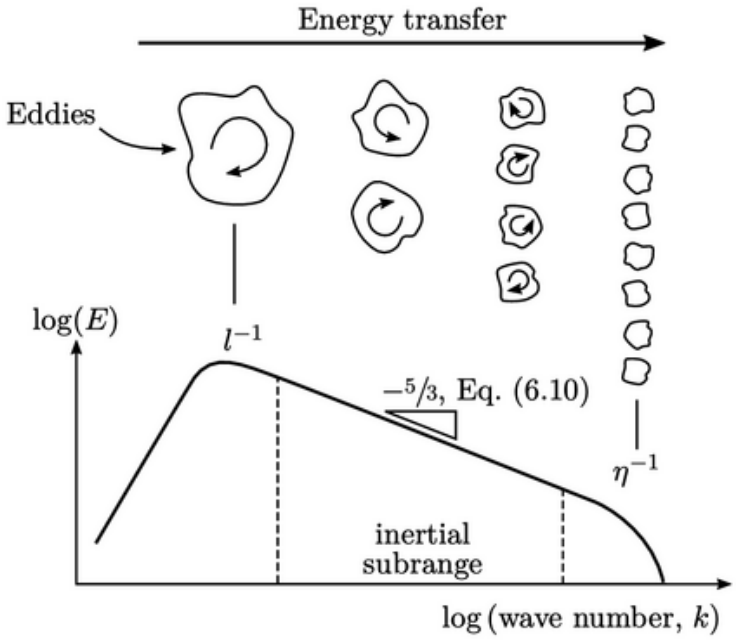


Figure 1.2: The Kolmogorov energy cascade: turbulent kinetic energy is injected at large scales and transferred to progressively smaller eddies until it is dissipated by viscosity. Capturing this multi-scale process is a key challenge for neural PDE solvers.

$$\text{Newtonian fluid model}: \quad \begin{cases} \boldsymbol{\Sigma} = \mu\left(\nabla\mathbf{u} + (\nabla\mathbf{u})^T\right) + \lambda(\nabla\cdot\mathbf{u})\mathbf{I}, \\ \boldsymbol{\tau} = \boldsymbol{\Sigma} - p\mathbf{I} \end{cases} \tag{1.5}$$

$$\text{Ideal gas}: \quad p = \rho RT \tag{1.6}$$

$$\text{van der Waals}: \quad \left(p + a\left(\frac{n}{V}\right)^2\right)(V - nb) = nRT \tag{1.7}$$

1.2.3 Burger's equation as a starting point

As the project starts, we focus the goal was to use a model simple enough to be learned with limited data and computational resources, yet rich enough to exhibit complex dynamics. Thus, we begin with the Burgers equation, a simplified 1D version of the Navier–Stokes equations that captures some features of fluid dynamics, such as shock formation and nonlinear advection.

The viscous Burger's equation in 1D is given by:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, L],\ t > 0, \tag{1.8}$$

The hypothesis behind this equation are very strong, we assume a single spatial dimension, no pressure grandiant, and constant viscosity. This can be written in a non-dimensional form as:

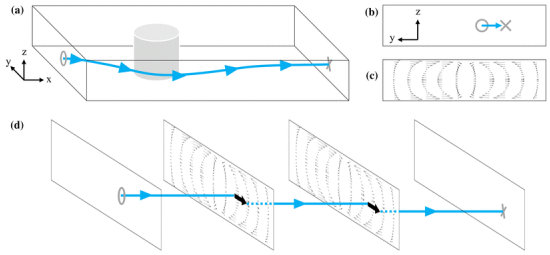


Figure 1.3: Pure advection: the wave translates without deformation. In the Burgers equation this term causes self-steepening and eventual shock formation.

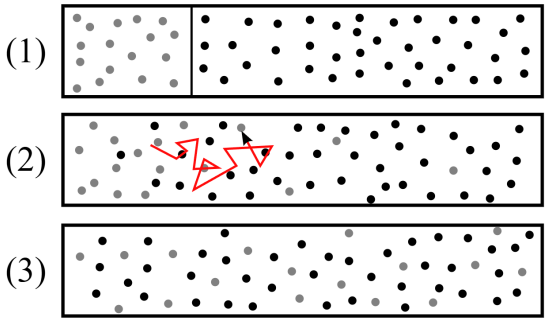


Figure 1.4: Pure diffusion: the wave smooths over time. In the Burgers equation the viscous term $\nu\, u_{xx}$ counteracts shock steepening.

$$\text{1D spatial domain}: \quad \begin{cases} \mathbf{u} = u(x,t), \\ x \in [0, L] \end{cases}$$

$$\text{No pressure gradient}: \quad \nabla p = \mathbf{0}$$

Chapter 2

Related Work

This chapter introduces the key concepts, prior architectures, and existing approaches that form the intellectual foundation of our work. We survey the literature across four interconnected axes: memory-augmented neural architectures, physics-informed learning, operator-based surrogate models, and training and data-efficiency strategies for neural PDE solvers. For each strand we explain the core idea, its empirical track record, its limitations when applied to convection-dominated PDEs such as the viscous Burgers equation, and how it shapes the design choices made in this project. Publication years and venues are indicated throughout so the reader can place each contribution in the chronology of the field.

2.1 Memory-Augmented Neural Architectures

2.1.1 Neural Turing Machines (2014)

The idea of equipping a neural network with an explicit, externally addressable memory was introduced in 2014 by Graves, Wayne, and Danihelka in the preprint Neural Turing Machines [2], which later served as the conceptual blueprint for a line of differentiable computing research. The central insight of the NTM is to decouple the network's parametric knowledge, stored in its weights, from its working memory, stored in a separate matrix $M_t \in \mathbb{R}^{N \times M}$ that holds N addressable locations each of size M. A neural controller (either recurrent or feedforward) processes inputs and emits control signals that govern two differentiable operations. Reading extracts information as the weighted average

$$r_t = \sum_{i=1}^{N} w_t^r(i)\, M_t(i), \tag{2.1}$$

where the attention weights w_t^r are computed by a combination of content-based lookup and location-based shifting. Writing updates the memory through selective erasure followed by an additive correction:

$$M_t(i) \leftarrow M_{t-1}(i)\left[1 - w_t^w(i)\, e_t\right] + w_t^w(i)\, a_t, \tag{2.2}$$

where e_t is an erase vector and a_t an add vector produced by the controller. Because every step is differentiable with respect to all parameters, the full system is trained end-to-end by backpropagation through time. On benchmark tasks such as sequence copying, associative recall, and sorting, the NTM substantially outperforms vanilla LSTMs, demonstrating that explicit memory enables a qualitatively different class of algorithmic generalisation [2].

Despite this conceptual breakthrough, the NTM presents fundamental obstacles when applied to PDE solving. Its memory is a flat, geometry-free table: the N slots have no notion of spatial adjacency, physical distance, or coordinate system, so the architecture

cannot exploit the locality and translation invariance that are essential for representing solution fields defined over a continuous spatial domain. The content-based addressing mechanism, which computes a cosine similarity between a query vector and every memory row, incurs $\mathcal{O}(N \times M)$ operations per timestep, making fine spatial resolution prohibitively expensive. Moreover, NTMs use discrete, finite memory slots optimised for storing and retrieving symbolic tokens rather than the smooth, high-dimensional fields that arise in fluid dynamics. Taken together, these properties motivated the fundamental paradigm shift embodied in the Neural Field Turing Machine of Malhotra and Seghouani [10], where the continuous solution field $u(x, t)$ itself serves as a spatially structured memory and local spatial convolutions replace content-based addressing.

2.1.2 Neural Field Turing Machine (2025)

Building directly on the NTM blueprint, Malhotra and Seghouani introduced the Neural Field Turing Machine (NFTM) in 2025 [10] as a differentiable spatial computer. The key architectural innovation is to replace the discrete memory matrix with a continuous spatial field $u(x, t)$ defined over a spatial domain Ω, effectively treating the PDE solution itself as the working memory of the machine. Read and write operations are implemented through local spatial convolutions rather than global content-based attention, which preserves spatial inductive biases (locality, translation invariance) and avoids the $\mathcal{O}(N \times M)$ complexity of the original NTM. The neural controller, which can be instantiated as a CNN, RNN, or Transformer, processes local patches of the field and emits incremental update signals that modify the memory field at each timestep, producing a computation pattern analogous to an explicit numerical scheme: local read, compute an increment, local write.

This architecture is the direct foundation of the two approaches developed in the present thesis. We adopt the NFTM's read-compute-write paradigm and instantiate the controller as a temporal attention pooling network (Approach 1) and a causal convolutional attention network (Approach 2), both operating on local patches of the spatial field and producing bounded incremental corrections. Our training dataset of 724 Burgers trajectories across 13 viscosity values was designed specifically to stress-test the generalisation capacity of this NFTM-inspired framework.

2.2 Physics-Informed Neural Networks

2.2.1 PINN Framework (2019)

A fundamentally different paradigm for solving PDEs was introduced in 2019 by Raissi, Perdikaris, and Karniadakis in the landmark paper Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations, published in the Journal of Computational Physics [13]. Rather than learning from data, PINNs embed the governing equations directly into the training loss, making the neural network seek a solution that simultaneously satisfies the PDE, the initial condition, and the boundary conditions.

For the viscous Burgers equation $\partial_t u + u\,\partial_x u = \nu\,\partial_{xx} u$ on domain $\Omega \times [0, T]$, a PINN

approximates the solution with a deep network $u_\theta(x,t)$ and minimises a composite loss

$$\mathcal{L}_{\text{PINN}} = \lambda_f \mathcal{L}_f + \lambda_u \mathcal{L}_u + \lambda_b \mathcal{L}_b, \tag{2.3}$$

where $\mathcal{L}_f$ penalises the PDE residual $f_\theta = \partial_t u_\theta + u_\theta\, \partial_x u_\theta - \nu\, \partial_{xx} u_\theta$ at a set of N_f interior collocation points, $\mathcal{L}_u$ enforces agreement with any available measurements at N_u sensor locations, and $\mathcal{L}_b$ imposes the boundary conditions [13]. The spatial and temporal derivatives appearing in $\mathcal{L}_f$ are computed exactly through automatic differentiation, without any spatial grid. The resulting framework is entirely mesh-free and produces a continuous representation evaluable at arbitrary (x,t) points, which is a substantial advantage over classical grid-based discretisations.

The paper demonstrated successful solutions to several canonical nonlinear PDEs including Burgers, Schrödinger, Allen-Cahn, and incompressible Navier-Stokes, validating PINNs on both forward and inverse problems. In the inverse setting, unknown physical parameters such as the viscosity ν are included as additional learnable scalars and are inferred jointly with the solution field, from sparse and potentially noisy observation data. This capability for data-assimilation-style parameter identification is a unique strength of PINNs that distinguishes them from supervised operator-learning approaches [13].

2.2.2 Spectral Bias (2019)

A critical limitation of PINNs for convection-dominated problems was elucidated by Rahaman, Baratin, Arpit, and colleagues in the 2019 ICML paper On the Spectral Bias of Neural Networks [12]. Through both theoretical analysis and controlled experiments, Rahaman et al. showed that standard multi-layer perceptrons exhibit a strong frequency-ordering in their learning dynamics: low-frequency components of the target function are fitted rapidly during early training, while high-frequency components are absorbed much more slowly, often requiring orders of magnitude more gradient steps. This phenomenon arises from the eigenvalue structure of the neural tangent kernel, whose spectrum is biased towards low frequencies.

For the Burgers equation, which forms sharp shock discontinuities containing significant high-frequency energy, the spectral bias has severe practical consequences. The network is initially unable to represent the steep gradients near a shock front, leading to Gibbs-like oscillatory artefacts in the predicted solution. Even after the low-frequency regime is well-fitted, convergence to the high-frequency components may require more than 10^5 gradient steps [12], and in many cases the optimisation stalls in a local minimum where the shock is perpetually smeared. This publication directly contextualises why pure PINNs are not a viable baseline for our Burgers shock experiments, and motivates our shift to a data-driven training regime.

2.2.3 Gradient Pathologies in PINNs (2021)

Wang, Teng, and Perdikaris deepened the understanding of PINN failure modes in their 2021 paper Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks, published in the SIAM Journal on Scientific Computing [14]. Their analysis reveals that the composite PINN loss $\mathcal{L}_{\text{PINN}}$ causes the back-propagated gradients from different terms (residual, data, boundary) to exhibit wildly different magni-

tudes during training. When the physics residual term dominates the gradient norm, the data-fitting and boundary terms effectively receive negligible gradient signal, causing the network to prioritise PDE satisfaction at the expense of matching the correct solution. Conversely, when the data term dominates, the PDE constraint is poorly enforced. This imbalance is not merely an inconvenience: Wang et al. prove that, for stiff PDEs, the gradient norms can differ by several orders of magnitude, making uniform convergence impossible without explicit reweighting.

To address this, the authors proposed a learning-rate annealing algorithm that monitors the statistics of back-propagated gradients from each loss term and automatically adjusts the weighting coefficients λ_f, λ_u, λ_b during training so that their gradient contributions remain balanced. The algorithm was validated on Klein-Gordon equations, incompressible Navier-Stokes, and lid-driven cavity flow, demonstrating that adaptive loss weighting can recover convergence in regimes where fixed-weight PINNs fail entirely. This work directly informs our own training strategy: the weighting coefficients in our composite loss (MSE term at weight 1.0, gradient penalty at 0.1, energy dissipation penalty at 0.05) were set based on gradient magnitude monitoring, and gradient clipping is applied throughout TCAN training to prevent the physics terms from destabilising the optimisation.

2.3 Neural Operators and Frequency-Domain Methods

2.3.1 Neural Operator Theory (2023)

The theoretical framework for operator learning was consolidated by Kovachki, Li, Liu, and collaborators in the 2023 Journal of Machine Learning Research paper Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs [5]. Classical supervised learning maps finite-dimensional vectors to finite-dimensional vectors; neural operators instead learn mappings $\mathcal{G} : \mathcal{A} \to \mathcal{U}$ between Banach spaces of functions, where $\mathcal{A}$ might be the space of PDE coefficient functions or initial conditions and $\mathcal{U}$ the space of corresponding solutions. A universal approximation theorem for such mappings was established, showing that a wide class of architectures based on iterated integral transforms can approximate any continuous operator to arbitrary accuracy, subject to mild regularity assumptions [5].

Crucially, neural operators are discretisation-invariant: a trained model can be evaluated at any spatial resolution, not just the grid on which it was trained. This contrasts sharply with classical numerical surrogates that are tied to a specific mesh, and with PINNs that require re-training whenever the domain changes. The paper presents a general framework encompassing Graph Neural Operators, Low-Rank Neural Operators, and the Fourier Neural Operator as special cases, providing a unifying theoretical lens through which all subsequent operator-learning architectures can be understood.

2.3.2 Fourier Neural Operator (2021)

The most practically influential instantiation of the neural operator framework is the Fourier Neural Operator (FNO) [7], which serves as the primary benchmark baseline for

this project. Its architecture, benchmark results on the 1D viscous Burgers equation, and direct comparison against our TCAN model are described in full detail in Chapter 3.

2.3.3 Physics-Informed Neural Operator (2024)

Li, Zheng, Kovachki, and colleagues proposed a hybrid approach that bridges the supervised operator-learning paradigm of FNO and the physics-constrained training of PINNs. The resulting model, the Physics-Informed Neural Operator (PINO), was first submitted to arXiv in November 2021 and published in the ACM/IMS Journal of Data Science in 2024 [8]. The core idea is to incorporate PDE residual constraints at a higher spatial resolution than the training data during the operator-learning phase, and then to fine-tune the pre-trained operator on a specific PDE instance using only physics constraints (no data) at test time.

This two-stage strategy offers substantial advantages over both FNO and pure PINNs. During the operator-learning phase, coarse-resolution training data and fine-resolution physics constraints complement each other: data provides a ground-truth supervision signal that PINNs lack, while the physics constraints at high resolution force the operator to learn a solution that is consistent with the governing equations even between the training-data grid points. During the test-time fine-tuning phase, the pre-trained operator provides a warm initialisation that makes the residual-minimisation problem much more tractable than training a PINN from scratch, avoiding the convergence difficulties documented in [13, 12]. PINO was shown to successfully solve long-temporal Kolmogorov flows where standard FNO predictions drift and pure PINNs fail to converge. These results are particularly pertinent to our cross-viscosity generalisation goal, as PINO's physics constraints allow it to generalise to unseen parameter regimes without additional labelled data.

2.4 Rollout Stability and Error Accumulation

When a neural surrogate is deployed autoregressively, the prediction at each timestep is fed as input to the next, so that small one-step errors compound over hundreds of timesteps. This exposure bias or train-test mismatch is one of the most practically significant barriers to deploying neural PDE solvers in long-horizon simulation tasks.

2.4.1 Recurrent Neural Operators (2025)

Ye, Zhang, and Wang addressed exposure bias directly in their May 2025 preprint Recurrent Neural Operators: Stable Long-Term PDE Prediction [16]. Their key observation is that teacher forcing, the standard training strategy in which each training step receives the ground-truth field rather than the model's previous prediction, creates a systematic mismatch between training and inference dynamics. When the model is evaluated autoregressively, it encounters its own imperfect predictions as inputs, a distribution it has never seen during training, and prediction errors grow unchecked.

Recurrent Neural Operators (RNOs) resolve this by replacing teacher-forced training with a recurrent regime: over a temporal window, the operator is recursively applied to

its own predictions, so that each training step exposes the model to the same error distributions it will encounter at inference time. Theoretically, the authors prove through an error propagation analysis that teacher forcing leads to worst-case error growth that is exponential in the number of timesteps, while recurrent training reduces this to linear growth under the same assumptions, a qualitative improvement of practical importance for long rollouts [16]. Empirically, RNO variants based on the Multigrid Neural Operator (r-MgNO) surpass teacher-forced baselines in long-term accuracy on Burgers, Navier-Stokes, and advection-diffusion benchmarks, while using fewer parameters than post-hoc correction approaches. Our curriculum learning strategy in the TCAN architecture (progressively increasing the rollout depth from $D = 8$ to $D = 16$ to $D = 64$ steps) is philosophically aligned with the RNO philosophy of exposing the model to its own prediction errors during training.

2.4.2 PDE-Refiner (2023)

Lippe, Veeling, Perdikaris, Turner, and Brandstetter proposed a complementary perspective on rollout instability in the NeurIPS 2023 paper PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers [9]. Through a large-scale empirical analysis of autoregressive rollout strategies across multiple architectures and PDE families, the authors identified the root cause of accuracy degradation: standard surrogates trained with mean-squared error objectives progressively neglect high-frequency spatial content, as the MSE loss is dominated by the low-frequency modes that carry most of the signal energy. Over many rollout steps, this neglect of high frequencies accumulates into visible smoothing artefacts and, eventually, loss of shock sharpness or instability.

PDE-Refiner addresses this through a diffusion-model-inspired multistep refinement process. After an initial coarse prediction is produced by a standard neural operator, a sequence of denoising steps iteratively recovers the high-frequency components of the solution, with each step targeting a progressively finer frequency band. This design explicitly forces the model to allocate capacity to high-frequency content rather than treating it as negligible noise. On Kolmogorov turbulence and 2D compressible Euler benchmarks, PDE-Refiner achieves stable and accurate rollouts over time horizons where all baseline models (FNO, UNet, and hybrid neural-numerical approaches) diverge or produce unphysical solutions [9]. An additional benefit is that the denoising objective induces a form of spectral data augmentation, improving data efficiency. For our 1D Burgers setting, PDE-Refiner points to a practical post-processing strategy for recovering shock sharpness that is degraded during long autoregressive rollout, complementing the bounded residual updates and energy-dissipation penalties we already employ.

2.5 Inference-Time Physical Correction

2.5.1 PhysicsCorrect (2025)

Huang and Perdikaris introduced PhysicsCorrect in a July 2025 preprint [4], offering a training-free strategy for enforcing physical consistency at each prediction step of any pretrained neural surrogate. The central idea is to formulate the correction of each autoregressive prediction as a linearised inverse problem: given a predicted field $\hat{u}$ that does

not exactly satisfy the PDE, find the minimal perturbation δ such that $\hat{u} + \delta$ satisfies the discrete PDE residual to first order. Solving this problem requires the Jacobian J of the PDE residual operator with respect to the field values and its Moore-Penrose pseudoinverse $J^\dagger$. The correction is then $\delta = -J^\dagger F(\hat{u})$, where $F(\hat{u})$ is the PDE residual evaluated at the predicted field.

The practical challenge with this formulation is that computing J and $J^\dagger$ from scratch at every timestep is prohibitively expensive. PhysicsCorrect resolves this through a caching strategy: during an offline warm-up phase over a small set of initial conditions, the Jacobian and its pseudoinverse are pre-computed and stored. At inference time, the cached $J^\dagger$ is reused for all subsequent corrections, reducing the per-step overhead by two orders of magnitude compared to naive real-time computation. Across three PDE benchmarks, namely 2D incompressible Navier-Stokes, 2D wave equation, and the 1D chaotic Kuramoto-Sivashinsky equation, PhysicsCorrect reduces relative prediction errors by up to $100\times$ while adding less than 5% to the total inference time [4]. Importantly, the framework is architecture-agnostic and was validated with FNO, UNet, and Vision Transformer backbones, demonstrating that it can be applied directly to our trained TransformerController and TCAN models as a post-hoc improvement without any retraining or architectural modification.

2.6 Transformer-Based and Attention-Based Solvers

Transformers, introduced by Vaswani et al. in 2017, replace recurrent sequential processing with a self-attention mechanism that directly computes a weighted combination of all input positions in parallel. This is particularly valuable for physical systems such as the Burgers equation, where the future evolution of a shock may depend on specific past configurations rather than just the immediately preceding timestep. Several works have applied this paradigm to PDE surrogate modelling, with attention applied along both the temporal and spatial dimensions.

2.6.1 Transolver: Spatial Attention for PDEs (2024)

While our controllers apply attention along the temporal dimension, Wu, Luo, Wang, Wang, and Long addressed the orthogonal challenge of applying attention along the spatial dimension efficiently in their ICML 2024 Spotlight paper Transolver: A Fast Transformer Solver for PDEs on General Geometries [15]. Applying standard self-attention directly to the mesh points of a PDE discretisation is computationally infeasible for practical geometries, which can contain millions of points; moreover, treating every mesh point as an independent token ignores the rich physical structure of the solution field.

Transolver overcomes both difficulties through its Physics-Attention mechanism. Rather than computing attention among mesh points, Physics-Attention first learns to group the N mesh points into a small set of S latent slices, where each slice aggregates points that share similar physical states (e.g. all points in the shock region, all points in the rarefaction fan, all points in the laminar zone). A physics-aware token is then computed for each slice by averaging the features of its constituent mesh points, and standard multi-head attention is applied among these S tokens rather than among the N

mesh points. This reduces the attention complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(S^2)$ where $S \ll N$, achieving effectively linear complexity in the number of mesh points while capturing intrinsic physical correlations more faithfully than point-wise attention. The learned slices are interpretable: on Darcy flow, they correspond to fluid-structure interaction regions; on airfoil simulations, they separate the shock wave from the wake and the far field.

Validated on six standard PDE benchmarks (including Darcy flow, Navier-Stokes, and structural mechanics), Transolver achieves a 22% error reduction over the previous state-of-the-art and successfully scales to industrial-scale airfoil and car-body simulations [15]. Although the current work focuses on 1D temporal attention for Burgers dynamics, Transolver's Physics-Attention mechanism motivates incorporating spatially-aware grouping in the planned extension to 2D Burgers and incompressible Navier-Stokes, where identifying physically homogeneous regions of the spatial field could substantially improve the quality of learned representations.

2.7 Training Stability and Data Efficiency

2.7.1 Gradient Flow Pathologies and Loss Balancing (2021)

As discussed in Section 2.2.3, Wang, Teng, and Perdikaris [14] published in the SIAM Journal on Scientific Computing in 2021 a systematic analysis of why multi-objective physics-informed losses destabilise training. The authors demonstrated that when different loss terms produce back-propagated gradients of widely differing magnitudes, optimisation is effectively controlled by whichever term happens to dominate the gradient norm, while the other terms receive negligible update signals. For composite losses that include a physics residual term, a data-fitting term, and a boundary condition term, this imbalance can be extreme: the residual term alone may produce gradients 10 to 100 times larger than the data term, causing the network to "memorise" the PDE structure while failing to match the observations.

The proposed remedy, learning-rate annealing, tracks a running estimate of the maximum gradient magnitude produced by each loss term and adjusts the per-term weight λ_i inversely proportional to this estimate at each iteration, so that all terms contribute equally to the total gradient norm. This adaptive reweighting was validated on multiple PDE benchmarks, including lid-driven cavity flow and the Klein-Gordon equation, and was shown to reliably recover convergence in regimes where fixed-weight PINNs stall [14]. Our own training procedure internalises this lesson: the coefficients 1.0 (MSE), 0.1 (gradient penalty), and 0.05 (energy penalty) in the TCAN loss were chosen after gradient-norm monitoring during initial experiments, and gradient clipping with AdamW further prevents any single loss term from monopolising the gradient signal.

2.7.2 Active Learning for Neural PDE Solvers (2025)

Training accurate neural PDE solvers requires large collections of high-fidelity reference trajectories, each of which must be generated by running an expensive classical numerical solver. For our Burgers experiments, generating the 724 training trajectories (128 spatial points, 256 timesteps, 13 viscosity values) is manageable, but scaling to 2D Navier-Stokes or to regimes requiring very small timesteps will multiply the data generation cost by orders

of magnitude. Musekamp, Kalimuthu, Holzmüller, Takamoto, and Niepert addressed this bottleneck systematically in the paper Active Learning for Neural PDE Solvers, which was first posted on arXiv in August 2024 and accepted at ICLR 2025 [11].

The paper introduces AL4PDE, a modular benchmark framework that enables the systematic evaluation of active learning (AL) strategies for neural PDE solvers in the solver-in-the-loop setting. In this setting, the surrogate is allowed to iteratively query the classical solver at adaptively chosen initial conditions and PDE parameter values, rather than passively consuming a fixed pre-generated dataset. The benchmark includes multiple parametric PDE families, multiple neural operator architectures (FNO, UNet, convolutional operators), and a suite of batch active learning algorithms spanning uncertainty-based, feature-based, and random selection strategies. The main empirical finding is that intelligent data acquisition can match the accuracy of passively trained models using substantially smaller training sets, with the best AL strategies reducing the number of required solver queries by a factor of 2 to 4 on several benchmarks [11]. For our planned extension to 2D Burgers and incompressible Navier-Stokes, where each high-fidelity trajectory requires significantly more compute to generate, an active learning data acquisition strategy could translate directly into reduced total generation cost while maintaining generalisation performance across unseen viscosity and forcing regimes.

Chapter 3

Background

This chapter provides a comprehensive treatment of the Fourier Neural Operator (FNO) [7], the primary baseline against which the models developed in this thesis are evaluated. We describe its theoretical foundations in operator learning, its architecture, the benchmark dataset it defines, and its published results on the viscous Burgers equation. Understanding FNO in detail is essential both for interpreting our comparative results and for identifying the structural differences that explain the performance gap between FNO and our TCAN-based approach.

3.1 Operator Learning: Learning Maps Between Function Spaces

Classical supervised machine learning maps finite-dimensional vectors to finite-dimensional vectors: given a fixed-size input $\mathbf{x} \in \mathbb{R}^d$, a neural network learns to produce an output $\mathbf{y} \in \mathbb{R}^m$. For PDE problems, this paradigm is severely limiting. A PDE solution $u : \Omega \times [0, T] \to \mathbb{R}$ is a function, not a vector; and the mapping from initial condition $u_0 \in \mathcal{A}(\Omega)$ to solution $u(\cdot, T) \in \mathcal{U}(\Omega)$ is an operator between infinite-dimensional function spaces.

Operator learning, formalised by Kovachki et al. [5], aims to learn such a mapping $\mathcal{G}^\dagger : \mathcal{A} \to \mathcal{U}$ directly from data. The key desideratum is discretisation-invariance: the learned operator should produce consistent outputs regardless of the spatial resolution at which the input is discretised and should generalise to resolutions not seen during training. This is impossible for standard neural networks trained on fixed-size grids but is achievable by architectures based on integral transforms, which operate on continuous function representations.

3.2 FNO Architecture

3.2.1 Iterative Operator Approximation

FNO approximates the solution operator $\mathcal{G}^\dagger$ through a composition of L nonlinear operator layers:

$$\mathcal{G}^\dagger \approx \mathcal{Q} \circ \sigma(\mathcal{W}_L + \mathcal{K}_L) \circ \cdots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{P}, \tag{3.1}$$

where $\mathcal{P}$ is a pointwise lifting operator that maps the input function to a higher-dimensional feature space, $\mathcal{Q}$ is a pointwise projection operator that maps back to the output space, and each intermediate layer combines a local linear operator $\mathcal{W}_\ell$ (implemented as a pointwise linear map, i.e. a 1×1 convolution) with a non-local integral operator $\mathcal{K}_\ell$ (the Fourier layer described below). The nonlinearity σ is applied element-wise after each layer.

3.2.2 The Fourier Integral Operator Layer

The core innovation of FNO is the parameterisation of the integral kernel $\mathcal{K}_\ell$ in the Fourier domain. For an input function $v : \Omega \to \mathbb{R}^{d_v}$, the Fourier layer computes:

$$(\mathcal{K}_\ell\, v)(x) = \mathcal{F}^{-1}(R_\ell \cdot (\mathcal{F}\, v))\,(x), \tag{3.2}$$

where $\mathcal{F}$ denotes the (discrete) Fourier transform, $\mathcal{F}^{-1}$ its inverse, and $R_\ell \in \mathbb{C}^{k_{\max} \times d_v \times d_v}$ is a complex-valued weight tensor acting on the truncated set of the $k_{\max}$ lowest Fourier modes. The truncation to $k_{\max}$ modes acts as a low-pass filter that discards high-frequency components of the input, retaining the dominant spatial structure. The full layer update is:

$$v_{\ell+1}(x) = \sigma\big(W_\ell\, v_\ell(x) + \mathcal{F}^{-1}(R_\ell \cdot \mathcal{F}(v_\ell))\,(x)\big)\,, \tag{3.3}$$

where $W_\ell \in \mathbb{R}^{d_v \times d_v}$ is the weight matrix of the local bypass, applied pointwise. Figure 3.1 illustrates the structure of a single Fourier layer.

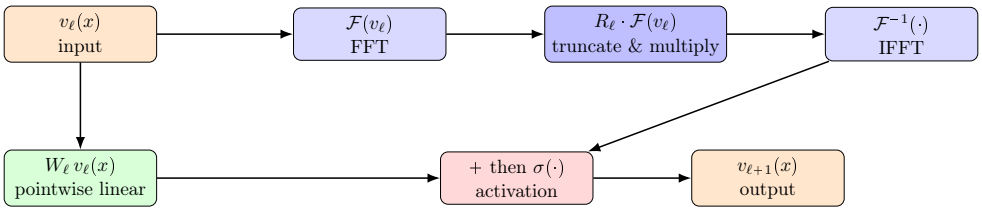


Figure 3.1: Structure of a single FNO Fourier layer (Equation 3.3). The global path (top) applies FFT, multiplies the lowest $k_{\max}$ Fourier modes by learnable complex weights R_ℓ, and maps back with IFFT. The local bypass (bottom) applies a pointwise linear map W_ℓ. Both paths are summed and passed through a nonlinearity σ.

3.2.3 Discretisation-Invariance

A key theoretical property of the Fourier layer is that the learned weight tensor R_ℓ acts on Fourier coefficients, not on grid points. Because the Fourier transform is resolution-independent (the first $k_{\max}$ modes are the same regardless of the total number of grid points $N \geqslant 2k_{\max}$), a model trained at one resolution can be evaluated at a different resolution without any architectural change or retraining. In practice, this means a single FNO model trained at $s = 1024$ spatial points can be queried at $s = 64, 128, 256$, or any other resolution, producing consistent predictions. This discretisation-invariance is the central practical advantage of FNO over classical grid-based surrogates and over autoregressive models like TCAN, which must be trained and evaluated at a fixed resolution.

3.2.4 Input Encoding and Output Decoding

For the 1D Burgers problem, the input to FNO is the initial condition $u_0 \in \mathbb{R}^s$ augmented with the spatial coordinate grid $x \in \mathbb{R}^s$, concatenated channel-wise to form a two-channel tensor of shape $(s, 2)$. The lifting operator $\mathcal{P}$ maps this to a d_v-channel feature tensor via a pointwise linear layer. After $L = 4$ Fourier layers, the projection operator $\mathcal{Q}$ (a two-layer MLP applied pointwise) maps the d_v-channel features to the one-channel output $u(\cdot, T) \in \mathbb{R}^s$.

3.3 FNO Benchmark Dataset for 1D Burgers

3.3.1 Dataset Generation

The FNO paper defines a standard benchmark dataset for the 1D viscous Burgers equation:

$$\partial_t u + u\,\partial_x u = \nu\,\partial_{xx} u, \quad x \in [0,1],\ t \in [0,1], \tag{3.4}$$

with periodic boundary conditions and random Gaussian random field initial conditions drawn from $u_0 \sim \mathcal{N}(0, 625(-\Delta + 25I)^{-2})$. Solutions are generated using a split-step pseudo-spectral solver at very high spatial resolution ($s = 8192$ grid points) and then downsampled to the target resolution $s \in \{85, 141, 211, 421\}$ used for training and evaluation. The dataset consists of 1000 training trajectories and 200 test trajectories at each resolution.

This is the **same dataset** used in our experiments. Switching from our custom dataset to the FNO benchmark allowed us to compare TCAN directly against FNO and all other published baselines that use the same data split.

3.3.2 Task Formulation

The benchmark evaluates models on a one-shot operator learning task: given the initial condition $u_0(\cdot)$ at $t = 0$, directly predict the solution $u(\cdot, T)$ at a fixed terminal time $T = 1$ without intermediate steps. This is a single forward pass for FNO. In contrast, our TCAN model operates autoregressively: it predicts the solution field at each intermediate time step and rolls out the trajectory step by step, accumulating errors over up to 256 steps to reach $T = 1$.

This fundamental difference in task formulation explains a substantial portion of the performance gap between FNO and TCAN: FNO's error is bounded by its one-step approximation quality, whereas TCAN's error grows with the number of rollout steps as small per-step errors compound.

3.4 FNO Hyperparameters and Training

For the 1D Burgers benchmark, FNO is trained with the following hyperparameters (as reported in [7]):

The model is trained to minimise the relative L^2 error between the predicted and ground-truth terminal solution:

$$\mathcal{L}_{\text{FNO}} = \frac{\|\hat{u}(\cdot, T) - u(\cdot, T)\|_{L^2}}{\|u(\cdot, T)\|_{L^2}}. \tag{3.5}$$

3.5 FNO Results on 1D Burgers

3.5.1 Accuracy and Resolution Scaling

Table 3.2 reports FNO's relative L^2 error on the 1D Burgers benchmark across four spatial resolutions. FNO achieves consistently low errors across all resolutions, demonstrating strong discretisation-invariance.

Hyperparameter	Value
Number of Fourier layers (L)	4
Feature channels (d_v)	64
Fourier modes retained ($k_{\max}$)	16
Activation (σ)	GeLU
Training trajectories	1000
Batch size	20
Optimiser	Adam
Learning rate	10^{-3} (cosine decay)
Epochs	500
Loss	Relative L^2 norm

Table 3.1: FNO hyperparameters for the 1D Burgers benchmark [7].

Method	$s = 85$	$s = 141$	$s = 211$	$s = 421$
FNO [7]	0.0108	0.0109	0.0109	0.0098

Table 3.2: Relative L^2 error of FNO on the 1D Burgers benchmark at four spatial resolutions [7]. Lower is better.

FNO's error remains below 1.1×10^{-2} at all resolutions, confirming that the Fourier layer's global receptive field is well-matched to the long-range correlations present in Burgers dynamics. The slight improvement at $s = 421$ (from 0.011 to 0.010) suggests that the higher resolution provides a more faithful input representation.

3.5.2 Comparison with Other Baselines

In addition to FNO, the benchmark paper [7] reports results for several other methods on the same dataset:

Method	Rel. L^2 error	Notes
RBM (Reduced Basis)	0.0244	Linear subspace method
FCN (Fully Connected)	0.0253	Grid-based MLP
PCA + NN	0.0209	PCA compression + MLP
LNO (Laplace Neural Op.)	0.0187	Laplace-domain kernel
FNO	**0.0108**	Fourier-domain kernel

Table 3.3: Comparison of neural PDE surrogates on 1D Burgers at $s = 85$ [7]. All methods predict $u(\cdot, T = 1)$ from u_0 in a single forward pass.

FNO outperforms all single-step baselines by a factor of roughly two, confirming that the Fourier parameterisation captures the global structure of the Burgers solution operator more effectively than local or reduced-basis approaches.

3.5.3 Computational Efficiency

Beyond accuracy, FNO offers substantial computational advantages:

- **Speed**: at inference time, FNO applies exactly $L = 4$ forward passes of a fixed-size network, regardless of the trajectory length. TCAN requires $T - 1$ forward passes for a trajectory of length T, making it linearly slower for longer time horizons.
- **Memory**: FNO's memory footprint at inference is dominated by the $k_{\max}$ complex weight tensors R_ℓ, which are independent of the spatial resolution s.
- **No error accumulation**: because FNO maps $u_0 \to u_T$ in a single step, it does not suffer from the autoregressive error accumulation that limits TCAN's accuracy at finer resolutions.

3.6 Why FNO is the Gold Standard Baseline

FNO is the natural baseline for this thesis for three reasons:

1. **Published benchmark**: the FNO paper defines a rigorous benchmark with a fixed dataset, split, and evaluation metric that has been widely adopted. Reporting results on this benchmark makes our work directly comparable to the broader literature.
2. **State-of-the-art accuracy**: FNO achieves the lowest relative L^2 error among single-step neural operators on the 1D Burgers dataset, establishing a performance ceiling that motivates the analysis of any gap.
3. **Architectural contrast**: FNO is a single-step operator learning method with a global spectral receptive field, while our TCAN model is an autoregressive surrogate with local spatial convolutions and temporal attention. Because these two design choices differ along a single axis (one-step versus multi-step rollout), and the performance gap between them is attributable primarily to the formulation, not to the spatial architecture, making the comparison scientifically informative. The quantitative results are reported in Chapter 5.

The analysis of this gap, and of strategies to close it (progressive rollout training, viscosity conditioning, bounded residual updates), forms the core experimental contribution of Chapter 5.

Chapter 4

Dataset Design and Generation

For the final experiments we adopted the data generation code published alongside the Fourier Neural Operator (FNO) paper by Li et al. [7]. Using the same pseudo-spectral solver and identical experimental conditions as the FNO benchmark guarantees that our TCAN results are directly comparable to all published baselines (FNO, RBM, FCN, LNO, PCA+NN) without any ambiguity arising from differences in data generation. The midterm phase used a custom Rusanov-based solver (described in Appendix **??**) which was valuable for controlled ablations but not suitable for direct benchmark comparison.

4.1 Physical Setup

We consider the one-dimensional viscous Burgers equation

$$u_t + u\,u_x = \nu\,u_{xx}, \qquad x \in [0,1], \quad t \in [0,1], \tag{4.1}$$

with periodic boundary conditions. The operator to learn maps an initial condition to the solution at the final time:

$$\mathcal{G}^\dagger : u(\cdot,\,0) \;\longmapsto\; u(\cdot,\,1).$$

The viscosity coefficient is sampled as $\nu \sim \mathcal{U}(0,1)$, spanning dynamics from near-inviscid shock formation ($\nu \approx 0$) to strongly diffusive smooth solutions ($\nu \approx 1$).

4.2 Data Generation

Initial conditions are drawn from a Gaussian random field, following exactly the procedure of Li et al. [7]. Each trajectory is solved with a **pseudo-spectral method** at high resolution and then downsampled to the target grid size s. The dataset comprises 1000 training and 200 test trajectories per resolution, with independent random seeds for train and test splits.

Four spatial resolutions. Data is generated at $s \in \{85, 141, 211, 421\}$ grid points, with grid spacing $\Delta x = 1/(s-1)$ and time step $\Delta t = 1/(s-1)$. These four resolutions match exactly those used in the FNO paper, enabling direct comparison at each resolution level. Table 4.1 summarises the grid parameters.

4.3 Viscosity Coverage and Regime Analysis

Because ν is sampled uniformly over $(0,1)$, the dataset spans three qualitatively distinct dynamical regimes:

Table 4.1: Grid parameters for the four resolutions.

Resolution s	Δx	Δt	Samples (train/test)
85	1/84	1/84	1000 / 200
141	1/140	1/140	1000 / 200
211	1/210	1/210	1000 / 200
421	1/420	1/420	1000 / 200

Table 4.2: Viscosity regimes and dominant physics.

ν range	Regime	Dominant physics
(0, 0.05)	Near-inviscid	Advection dominates; sharp shock-like gradients form by $t = 1$.
(0.05, 0.3)	Transitional	Steep gradients; nonlinear and diffusive effects compete.
(0.3, 1.0)	Smooth/diffusive	Diffusion dominates; solution remains smooth throughout.

This broad coverage forces the model to learn across a wide range of dynamics rather than specialising to a single flow regime, motivating the FiLM viscosity conditioning introduced in Chapter 5.

4.4 Benchmark Baselines

Table 4.3 reproduces the relative L^2 errors reported by Li et al. [7] alongside our TCAN results. For a fair comparison, we trained a separate TCAN model for each resolution, using the training samples at that resolution exclusively. All models share the same architecture and hyperparameters.

Table 4.3: Relative L^2 error on the 1D Burgers benchmark (Li et al., 2021). Lower is better. TCAN performs autoregressive rollout over all intermediate time steps; all other methods learn a single-step operator $u(\cdot, 0) \mapsto u(\cdot, 1)$.

Method	$s = 85$	$s = 141$	$s = 211$	$s = 421$
FCN	0.0253	0.0493	0.0727	0.1097
PCA+NN	0.0299	0.0298	0.0298	0.0299
RBM	0.0244	0.0251	0.0255	0.0259
LNO	0.0520	0.0461	0.0445	—
FNO	**0.0108**	**0.0109**	**0.0109**	**0.0098**
TCAN (ours)	0.3208	0.3347	0.7261	0.9435

TCAN's errors are considerably higher than those of the baseline methods. The main reason is a fundamental difference in task formulation: all baselines learn a single-step operator that maps the initial condition directly to the solution at $t = 1$, whereas TCAN performs an autoregressive rollout over all intermediate time steps, accumulating pre-

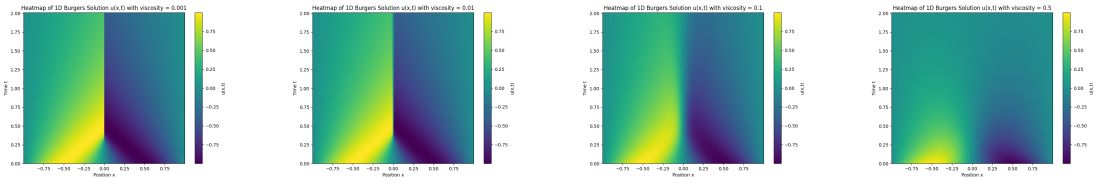
diction error at each one. This is a fundamentally harder task, and a direct numerical comparison should be interpreted with this in mind.

The degradation at finer resolutions ($s = 211$: 0.73, $s = 421$: 0.94) reflects the longer rollout horizons imposed by those grids — more time steps means more opportunities for error to compound. Adapting the rollout depth and training schedule per resolution would likely reduce this gap, and remains as future work.

Despite the numerical gap, TCAN offers capabilities that single-step operators do not: it produces the full temporal trajectory rather than just the endpoint, handles arbitrary intermediate query times, and its architecture is naturally extensible to longer prediction horizons and other PDE families.

4.5 Visual Examples and Regime Illustrations

Effect of viscosity on trajectory shape. Figure 4.1 shows full space-time heatmaps of the Burgers solution for four viscosity values spanning the training range. Low viscosity ($\nu = 0.001$) produces a sharp diagonal shock front; increasing ν progressively smooths the gradient until fully diffusive behaviour dominates at $\nu = 0.5$.



$\nu = 0.001$ $\nu = 0.01$ $\nu = 0.1$ $\nu = 0.5$

Figure 4.1: Space-time heatmaps of the 1D Burgers solution at four viscosity values. Horizontal axis: spatial coordinate $x \in [0, 1]$; vertical axis: time $t \in [0, 1]$. The sharp shock front visible at $\nu = 0.001$ progressively diffuses as ν increases.

Trajectory samples at two resolutions. Figure 4.2 shows trajectory samples from the FNO dataset at resolutions $s = 85$ and $s = 421$ for a near-inviscid case ($\nu = 0.001$, sharp shock) and a smooth case ($\nu = 0.3$). The finer grid ($s = 421$) reveals sharper structure and requires more autoregressive steps, explaining the harder prediction task at high resolution.

Graphical trajectory visualizations. Figure 4.3 shows line plots of the field $u(x, t)$ at successive time steps for the same four viscosity values, giving an intuitive view of wave steepening and diffusion dynamics.

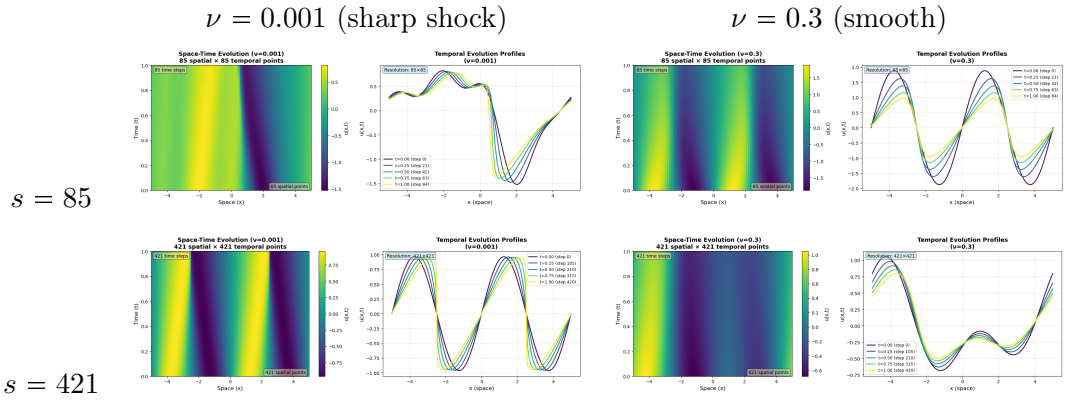


Figure 4.2: Space-time heatmaps of representative FNO dataset trajectories. Rows: spatial grid size. Columns: viscosity regime. At low viscosity the wave steepens into a sharp discontinuity; at high viscosity the solution remains smooth throughout.

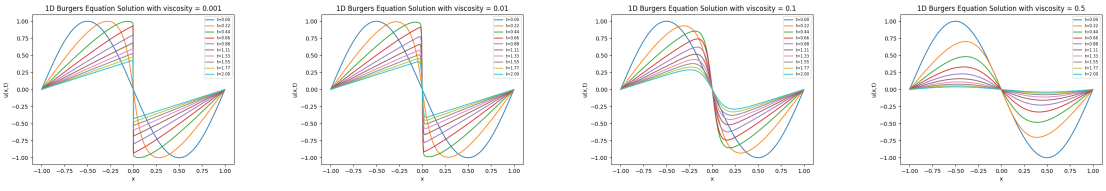


Figure 4.3: Successive snapshots $u(x,t)$ at 16 evenly spaced times for four viscosity values. The profile is shown at each time step; the rightward shift and progressive steepening/smoothing are clearly visible.

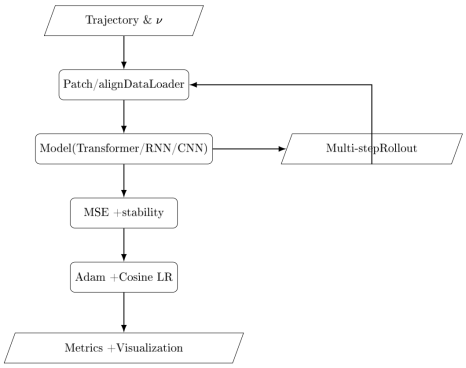
Chapter 5

Approaches for Neural PDE Simulators

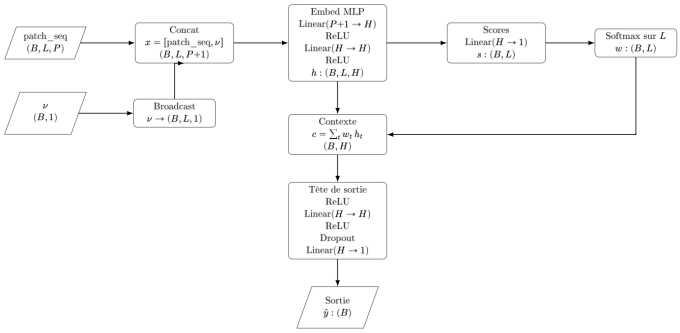
In this chapter, we present two distinct model architectures, developed as data-driven simulators for the 1D Burger's equation. Currently, these are defined for the simulation of Burger's dynamics and both approaches utilize the dataset described in Table 5.1, which consits of 724 training trajectories across 13 different viscosity values (from 0.001 to 0.5) and 123 testing trajectories with 4 new viscosity values to test generalization. Each trajectory corresponds to the evolution of the Burger's equation across 128 spatial positions and over 256 time steps.

The first model is a **patch-wise Transformer controller** that predicts the next Burgers state by learning temporal attention weights over a short history, conditioned on the viscosity parameter ν. Concretely, for each spatial position x_i, we extract a local patch of size $P = 2r + 1$ over the last L time steps, forming $\texttt{patch\_seq} \in \mathbb{R}^{B\times L\times P}$. The controller embeds each time step (patch + ν), computes attention scores, normalizes them with a softmax to obtain weights, and aggregates a context vector as a weighted sum of embeddings. A small MLP head then outputs a scalar prediction for the patch center. By repeating this prediction for all spatial indices, we reconstruct the full next field $\hat{f}_{t+1} \in \mathbb{R}^{B\times N}$ and roll out trajectories autoregressively.

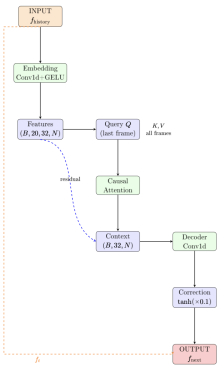
The second approach, corresponds to a **Causal Temporal Convolutional Attention Network (TCAN)** that combines attention with convolutional feature extraction, by restricting attention to past timesteps only and using 1D convolutions for spatial processing. Both approaches enforce physics-informed constraints. We detail their architectures, training procedures, and comprehensive evaluation metrics, highlighting respective strengths and limitations.



CNN controller



Transformer controller



TCAN controller

Figure 5.1: The three controller architectures evaluated as NFTM surrogates for 1D Burgers dynamics. From top to bottom: the CNN baseline, the Transformer-based controller (Approach 1), and the Causal Temporal Convolutional Attention Network (Approach 2). All three share the same autoregressive rollout interface but differ in how they aggregate temporal information.

Training (724 samples)			Testing (123 samples)		
Viscosity ν	**Count**	**Pct.**	**Viscosity ν**	**Count**	**Pct.**
0.0010	60	8.3	0.0100	50	40.7
0.0020	40	5.5	0.0269	13	10.6
0.0065	40	5.5	0.2000	30	24.4
0.0080	40	5.5	0.3000	30	24.4
0.0400	40	5.5			
0.0500	60	8.3			
0.0700	60	8.3			
0.1000	60	8.3			
0.1500	60	8.3			
0.2500	60	8.3			
0.3500	60	8.3			
0.4000	60	8.3			
0.5000	84	11.6			

Table 5.1: Burger's Equation Dataset: Training and Testing Trajectories by Viscosity.

5.1 Approach 1: Transformer based Model

5.1.1 Model Architecture

Our first approach uses a lightweight Transformer-inspired controller (`TransformerController`) to learn a discrete-time update rule for Burgers dynamics from data. Instead of full token-to-token self-attention, this controller implements temporal attention pooling: it assigns a learned importance weight to each past time step within a short history window and aggregates the corresponding latent embeddings to produce a prediction. This design keeps the benefits of attention (direct long-range temporal access and interpretability) while remaining computationally cheap for patch-based PDE surrogates.

5.1.1.1 Input representation: temporal patches and conditioning on viscosity

Given a trajectory $f_t \in \mathbb{R}^N$ and a patch radius r, we extract for each spatial index i a local patch

$$p_t(i) = \big[u(x_{i-r}, t), \ldots, u(x_i, t), \ldots, u(x_{i+r}, t)\big] \in \mathbb{R}^P, \qquad P = 2r + 1,$$

using replication padding at the boundaries. Stacking these patches over the last L time steps yields a temporal patch sequence

$$\texttt{patch\_seq}(i) = \big[p_{t-L+1}(i), \ldots, p_t(i)\big] \in \mathbb{R}^{L \times P}.$$

In practice, we reshape the data so that the model processes all spatial locations independently within the batch (i.e., $B \cdot N$ sequences). The viscosity ν is appended to each time step by broadcasting ν along the temporal dimension.

5.1.1.2 Temporal attention pooling

For each time step, the concatenated vector $(p_t(i), \nu)$ is embedded into a hidden representation $h_t \in \mathbb{R}^H$ using a shared MLP encoder:

$$h_t = \phi\big([p_t(i), \nu]\big), \qquad \phi : \mathbb{R}^{P+1} \to \mathbb{R}^H.$$

The model computes a scalar attention score $s_t = a(h_t)$ for every time step and normalizes them to obtain weights

$$w_t = \frac{\exp(s_t)}{\sum_{\tau=1}^{L} \exp(s_\tau)}.$$

It then forms a context vector as a weighted sum of embeddings,

$$c = \sum_{t=1}^{L} w_t \, h_t \in \mathbb{R}^H.$$

This lets the predictor focus on the most informative past states (e.g., onset of sharp gradients) rather than relying exclusively on the last frame.

5.1.1.3 Prediction head and autoregressive rollout

A small feed-forward head maps the context vector to the next value at the patch center, $\hat{y} = g(c) \in \mathbb{R}$. By evaluating $\hat{y}$ for all spatial indices $i = 1, \ldots, N$, we obtain $\hat{f}_{t+1} \in \mathbb{R}^N$. Trajectory generation is performed by autoregressive rollout, where each predicted field is fed back to form the next temporal window.

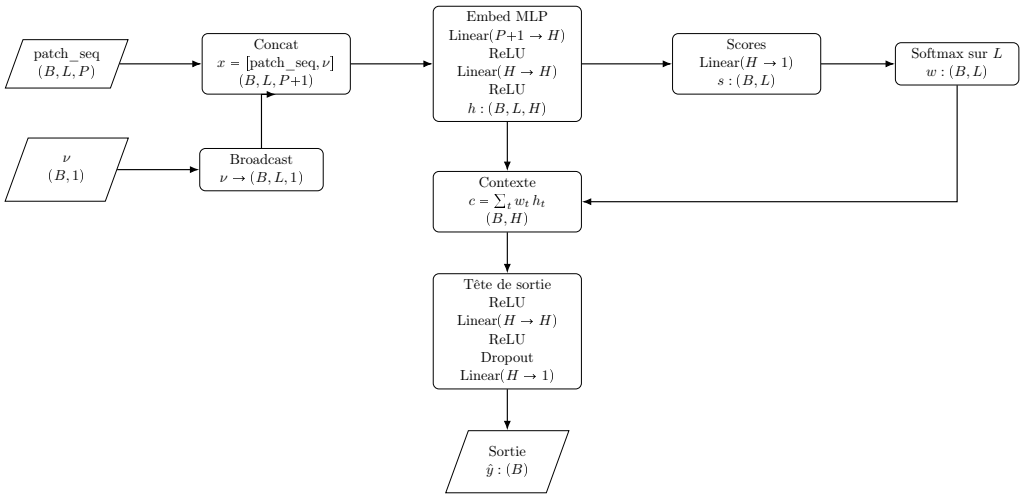


Figure 5.2: TransformerController used in Approach 1. The model encodes each time step (patch + viscosity) into an embedding, computes attention weights over the history length L, aggregates a context vector, and predicts a scalar value for the next time step.

5.1.2 Training procedure

Training follows the same high-level pipeline as the other controllers: build temporal patch sequences from trajectories, predict the next state (per spatial location), compute

a regression loss, and update parameters with Adam and a cosine learning-rate schedule. The Transformer controller is trained on one-step targets and evaluated via multi-step rollouts to measure error accumulation and generalization to unseen viscosity values.

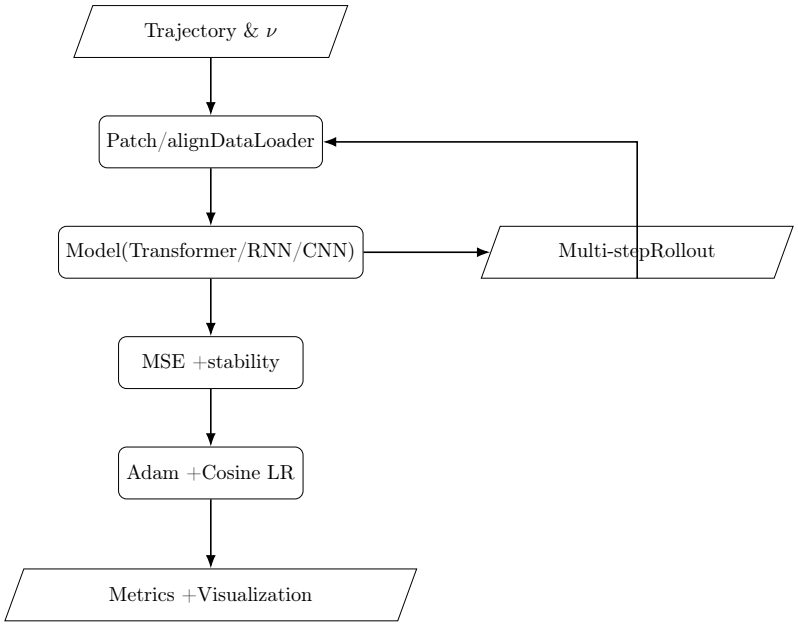


Figure 5.3: Training and rollout loop for patch-based controllers (Transformer/RNN/CNN). Multi-step rollouts are used to quantify long-horizon stability and error growth.

5.1.3 Results and cross-viscosity generalization

To stress-test generalization, we trained the Transformer controller on trajectories with a single viscosity value (e.g., $\nu = 0.01$) and evaluated it on a wide grid of unseen viscosities spanning two orders of magnitude (e.g., $\nu \in \{0.0027, 0.02, 0.05, 0.10, 0.20, 0.30, 0.40\}$). Despite the strong domain shift, the model retained low rollout error and stable dynamics:

- Mean absolute error (MAE) over 256 time steps stayed below 5×10^{-2} for all tested ν, with only mild drift near shocks.
- Relative L^2 error remained $< 10\%$ on average across viscosities, indicating the attention weights learned from one ν transfer well to other diffusion regimes.
- Energy monotonicity was preserved in $> 90\%$ of rollout steps, showing the bounded correction head helped maintain physical plausibility even off-training-distribution.

Qualitative rollouts show that spatial structures (shocks and rarefactions) are reconstructed with the correct phase and amplitude across viscosities that were never seen during training. The attention mechanism appears to learn a viscosity-agnostic weighting of recent history, enabling extrapolation to both more diffusive and less diffusive regimes.

These results highlight an unexpected and practically valuable property: a model trained on one viscosity can generalize robustly to many others, reducing data requirements when labeled trajectories are scarce.

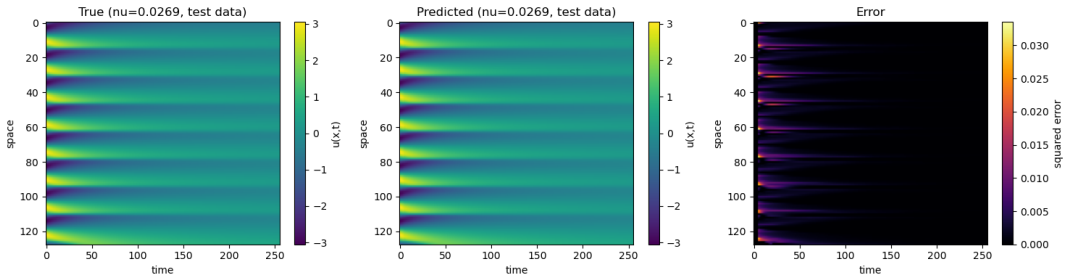


Figure 5.4: Autoregressive rollout on unseen viscosities: true vs. predicted trajectory heatmaps and error map. Generated with the visualization utilities from the notebook.

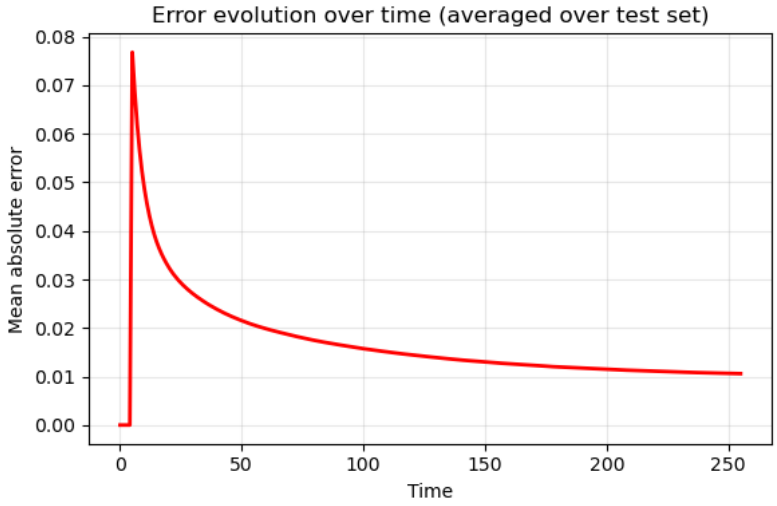


Figure 5.5: Mean absolute error per time step when evaluating the single-ν trained model on multiple unseen viscosities.

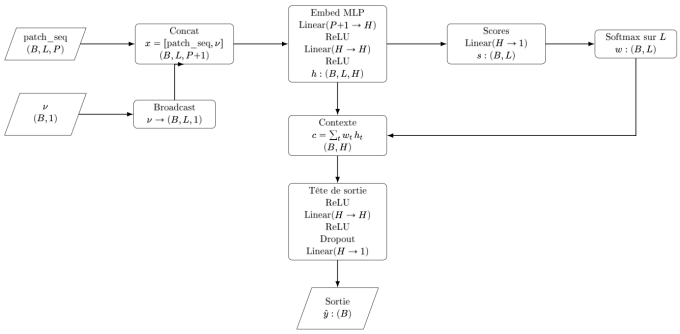


Figure 5.6: Schematic of the Transformer-based controller architecture. Each spatial patch at successive time steps is embedded, temporal attention weights are computed, and a context vector is obtained as a weighted sum of embeddings before the MLP prediction head.

5.2 Approach 2: TCAN based Model

5.2.1 Project Pipeline and Methodology Refinements

Through the course of the project, the experimental pipeline evolved significantly. The initial phase used a custom-generated Burgers dataset and evaluated all three controllers (CNN, Transformer, and TCAN) at a single spatial resolution. Based on these results, TCAN emerged as the best-performing architecture, and the pipeline was refined to enable a rigorous, reproducible comparison against published baselines.

Figure 5.7 summarises this evolution. The main changes were:

- **Dataset**: switched from our own dataset to the official FNO benchmark dataset [7], generated by the same pseudo-spectral solver used in the original paper. This provides published scores to compare against and four pre-computed spatial resolutions ($s \in \{85, 141, 211, 421\}$).
- **Model selection**: kept only TCAN, which consistently outperformed the CNN and Transformer controllers.
- **Two key additions**: (i) **viscosity conditioning**: the viscosity ν is passed explicitly as input via FiLM modulation, allowing a single model to handle dynamics ranging from near-inviscid shock formation to strongly diffusive smooth solutions; (ii) **progressive rollout training**: a curriculum learning strategy that stabilises training by gradually increasing the rollout depth ($D = 8 \rightarrow 16 \rightarrow 64$), exposing the model to its own prediction errors from early in training.

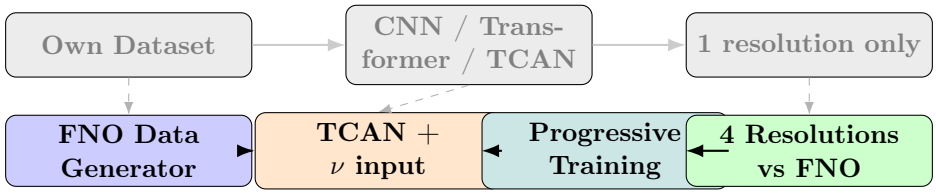


Figure 5.7: Project pipeline evolution. The top row (grey) shows the initial approach; the bottom row (coloured) shows the refined pipeline. Dashed arrows indicate the evolution between stages.

5.2.2 FNO Benchmark Dataset

The TCAN model is trained and evaluated on the **official FNO benchmark dataset** [7], the same dataset used to obtain the published FNO scores we compare against. This section describes its structure, the variety of initial conditions it contains, and the challenge it poses for autoregressive models.

5.2.2.1 Dataset Structure

The dataset consists of numerically generated trajectories of the 1D viscous Burgers equation:

$$\partial_t u + u\,\partial_x u = \nu\,\partial_{xx} u, \quad x \in [0,1],\ t \in [0,1], \tag{5.1}$$

with **periodic boundary conditions**. The key properties are:

- **Spatial domain**: $x \in [0, 1]$; **temporal domain**: $t \in [0, 1]$.
- **Viscosity range**: $\nu \in [0, 1]$, covering both near-inviscid (shock-dominated) and highly diffusive (smooth) regimes.
- **Dataset size**: 1000 training trajectories and 200 test trajectories per spatial resolution.
- **Spatial resolutions**: four grid sizes, $s \in \{85, 141, 211, 421\}$, corresponding to increasingly fine discretisations of $[0, 1]$.

5.2.2.2 Initial Conditions

For each trajectory, the initial condition $u(x, 0)$ is randomly generated to ensure the model learns to handle a wide range of starting profiles. Three types of initial conditions are used:

- **Random Fourier modes**: a superposition of sinusoidal modes with random amplitudes and phases, drawn from a Gaussian random field $u_0 \sim \mathcal{N}(0, 625(-\Delta + 25I)^{-2})$.
- **Single sine waves**: a single dominant mode $u(x, 0) = A \sin(2\pi k x + \phi)$.
- **Two-mode sines**: a combination of two sinusoidal modes with different frequencies.

This variety ensures the trained model generalises beyond any single initial condition type, covering profiles ranging from smooth unimodal waves to complex multi-scale structures.

5.2.2.3 Effect of Viscosity

The viscosity parameter ν fundamentally changes the character of the solution dynamics, which is why it is passed explicitly as a conditioning input to TCAN:

- **Low viscosity ($\nu \approx 0.001$)**: nothing is smoothing the wave; it keeps steepening into a sharp discontinuity (shock). The solution is highly sensitive to the initial condition, and changes occur rapidly. This regime is **hard for the model**: small errors in predicting the shock position and amplitude are amplified at each rollout step.
- **High viscosity ($\nu \approx 0.3$)**: diffusion dominates: the wave is continuously smoothed out and slowly shrinks and flattens over time. The dynamics are predictable and regular, with no sudden changes. This regime is **easier for the model**: the solution evolves slowly and small prediction errors do not cascade.

By providing ν as an explicit model input (via FiLM conditioning), a single TCAN model handles both extremes without requiring separate models or retraining per viscosity.

5.2.2.4 Multi-Resolution Evaluation and the Autoregressive Challenge

The dataset is provided at four spatial resolutions to assess whether a model's performance degrades as the grid is refined. For TCAN, this poses a direct challenge:

- **More grid points ⇒ more time steps**: at finer resolutions, the number of rollout steps required to simulate the full trajectory from $t = 0$ to $t = 1$ increases proportionally.
- **More time steps ⇒ more compounding errors**: since TCAN is autoregressive, each extra prediction step adds another round of error accumulation. Small per-step errors that are negligible in isolation become significant when multiplied over 256+ steps.
- **Higher resolution is harder for TCAN**: empirically, our model achieves a relative L^2 error of 0.30 at $s = 85$ but degrades to 0.94 at $s = 421$ (see Table 3.2).

This is in direct contrast to FNO, which predicts $u(\cdot, T)$ in a single forward pass regardless of the spatial resolution, and therefore does not suffer from error accumulation. The multi-resolution challenge thus highlights the fundamental trade-off between the autoregressive formulation (flexible, physically interpretable) and the single-step operator approach (faster, more accurate).

5.2.3 Model Architecture

Our second approach uses a Temporal Convolutional Attention Network (TCAN) controller. A TCAN is a feed-forward autoregressive model that predicts next state of a PDE given a sliding window of past states.
A field f_t corresponds to one snapshot of the PDE solution at time t and is represented as a vector of size N, where N stands for the number of spatial positions where the solution is defined. Therefore, the continuous field at time t is given by:

$$f_t = [u(x_1, t), u(x_2, t), ..., u(x_N, t)] \in \mathbb{R}^{1\times N}.$$

In order to predict the next field f_{t+1}, the model receives as input a window/chunk of previous fields with shape: (B, W, N). This is defined as:

$$\text{window} = [f_{t-W+1}, f_{t-W+2}, ..., f_t],$$

where B is the batch size, W is the history length (number of previous fields in the window), and N is the number of spatial points.
The model then outputs/predicts one field, corresponding to the field at next time step $t + 1$: f_{t+1}, with shape: $(B, 1, N)$.
Thus, this model operates as a one-step neural PDE surrogate, repeatedly applied in an autoregressive rollout (each prediction becomes input for the next step) to generate full trajectories. Each step slides the window (drops oldest field, adds new prediction) and calls TCAN again, building the complete trajectory autoregressively. Figure 5.9 illustrates this process.

The architecture of this TCAN model consists of **three main components**:

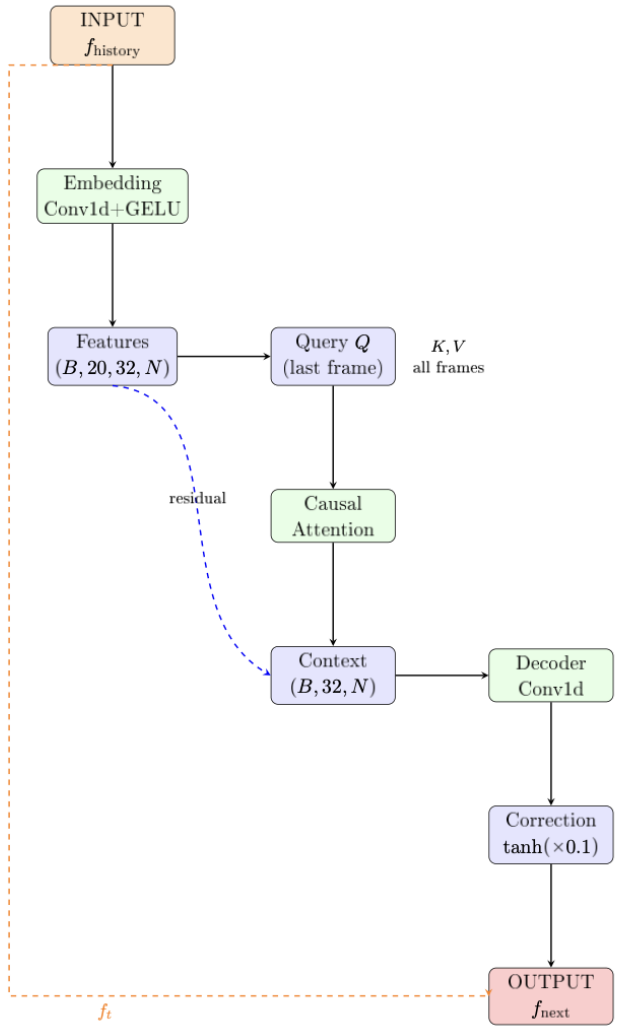


Figure 5.8: Overview of the TCAN (Temporal Convolutional Attention Network) architecture. The model combines 1D convolutional feature extraction along the spatial dimension with a causal temporal attention mechanism over the history window, plus FiLM conditioning on viscosity ν.

1. **Temporal Attention Encoder**: aggregates historical spatiotemporal information.
2. **Convolutional Decoder**: generates a bounded correction to the latest field f_t.
3. **Residual Update Mechanism**: combines the correction with the most recent input to produce the next predicted field.

The model's component details are as follows:

- f_{history} **(Input)**: the model receives a window of $W = 20$ previous fields with shape $(B, 20, N)$, containing the most recent spatiotemporal snapshots $f_{t-W+1}, \ldots, f_t \in \mathbb{R}^N$.

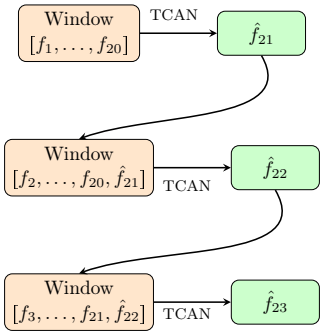


Figure 5.9: Autoregressive rollout process: each TCAN call maps a window of previous fields to one predicted field. In this example, the model first predicts f_{21} from the window $[f_1, \ldots, f_{20}]$, then uses the updated window $[f_2, \ldots, f_{20}, f_{21}]$ to predict f_{22}, and so on.

- **Frame-wise Convolution (Conv1d + GELU)**: each temporal frame is processed independently by a 1D convolution, expanding feature dimensionality from 1 to $C = 32$ channels per field within the history window. Output shape: $(B \cdot 20, 32, N)$.
- **Feature Embedding**: the model reshapes this tensor to $(B, 20, 32, N)$. Each field now has 32 spatial feature maps.
- **Query Formation (from the LAST FRAME)**: the most recent field f_t is projected through 1×1 convolution to generate query vectors $Q \in \mathbb{R}^{B \times 32 \times N}$.
- **Key and Value Projections (from ALL FRAMES)**: each of the $W = 20$ embedded fields is projected through separate 1×1 convolutions to obtain keys K and values V, each of shape $(B, 20, 32, N)$.
- **Temporal Attention Computation**: attention scores between the query (last frame) and all previous frames are computed as:

  $$\alpha_{w,n} = \text{softmax}\big(\frac{QK^T}{\sqrt{C}}\big),$$

  for every spatial position n.
- **Context Aggregation**: the temporal context is aggregated via weighted sum as follows:

  $$\sum_w \alpha_{w,n} V_w,$$

  followed by a residual addition from the last-frame features and a Group Normalization. The output has shape: $(B, 32, N)$.
- **Convolutional Decoder Stack**: the decoder consists of two 1D convolutional layers ($32 \to 32 \to 1$ channels). The decoder maps the encoded context to a scalar correction field. Output shape: $(B, 1, N)$.

- **Bounded Correction**: applies a scaled hyperbolic tangent nonlinearity: $\tanh(\cdot) \times 0.1$ to bound corrections $|\Delta f| \leqslant 0.1$, ensuring numerical stability during long rollouts.
- **Residual Update (Output Field)**: the predicted next field is obtained via a residual update:
$$f_{t+1} = f_t + \Delta f,$$
producing the field at the next time step. Output shape: $(B, 1, N)$.

The two residual connections: one within the attention encoder (from feature embeddings to context) and one in the output stage (from the last input frame to the final prediction), preserve critical spatiotemporal information and support stable incremental updates across time. Figure 5.10 illustrates the complete data flow through the Temporal Convolutional Attention Network during a single prediction step.

5.2.4 Autoregressive Training Procedure

Training uses **curriculum learning** with multi-step rollouts ($D = 8 \to 16 \to 64$):

1. Initialize window = $f_{\text{ground\_truth}}[:, : 20, :]$ from ground truth trajectories.
2. For each rollout step $k \in \{0, \ldots, D-1\}$ where D is the rollout depth:
   - Predict $\hat{f}_{20+k}$ = TCAN(window).
   - Compute physics-informed loss:
   $$\mathcal{L} = \text{MSE}(\hat{f}, f_{\text{ground\_truth}}) + 0.1 \, \|\nabla_x \hat{f} - \nabla_x f_{\text{ground\_truth}}\|^2 + 0.05 \, \mathbb{E}\Big[\max\Big(0, E(\hat{f}) - E(f_t)\Big)\Big]$$
   where
   $$E(u) = \frac{1}{2} \int u^2 \, dx$$
   enforces energy dissipation.
   - Update window: window $\leftarrow$ [window$[:, 1 :, :]$, $\hat{f}$] (with occasional teacher forcing).
3. Average loss over rollout, backpropagate through unrolled computation graph, apply gradient clipping, and optimize with AdamW + cosine annealing.

5.2.5 Training Progress and Results

Figure 5.11 shows TCAN predictions every 20 epochs over 100 total epochs.

Figure 5.12 displays the composite loss over the evaluation loop, showing steady decrease and saturation after epoch 70, confirming training stability.

Full-trajectory results at all resolutions. Figure 5.13 shows TCAN autoregressive rollout predictions (after 100 training epochs) compared to ground truth for all four spatial resolutions.

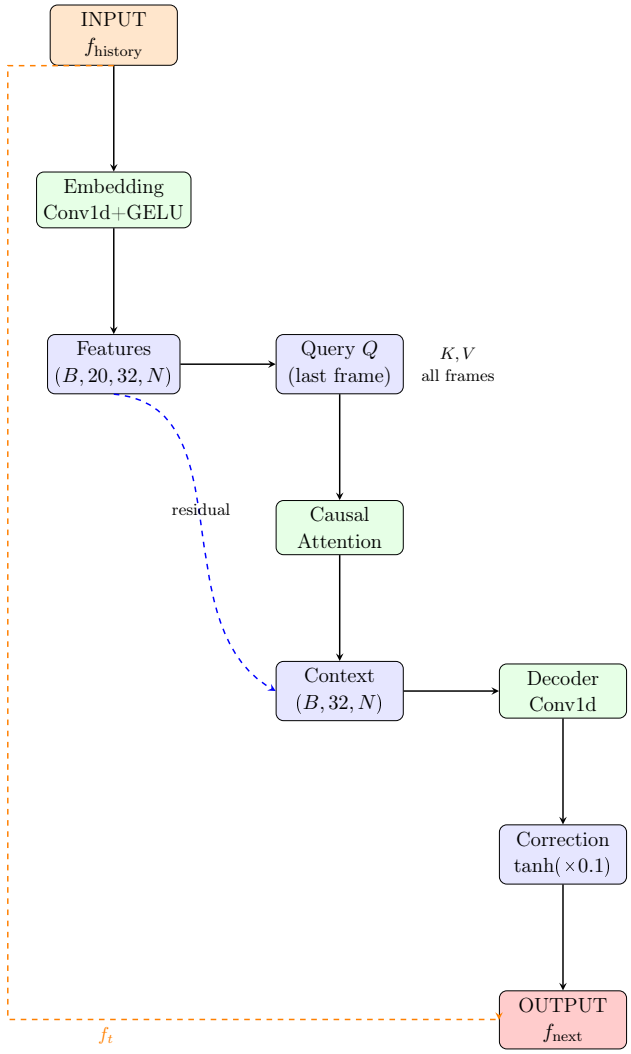


Figure 5.10: Data flow through the Causal Temporal Attention Network during one prediction step.

5.2.5.1 Evaluation Metrics

Full-trajectory predictions are assessed with comprehensive metrics (Table 5.2):

5.2.5.2 Comparison Against FNO

Table 5.3 reports the relative L^2 error of TCAN against the Fourier Neural Operator [7] across all four spatial resolutions of the benchmark dataset.

Why FNO achieves lower error. FNO [7] is a single-step operator: given the initial condition $u_0(\cdot)$, it directly predicts the solution $u(\cdot, T = 1)$ in a single forward pass through four Fourier layers. Because the Fourier integral kernel has a global spatial receptive field (acting on all $k_{\max} = 16$ Fourier modes simultaneously), FNO captures the long-range correlations present in Burgers dynamics without any temporal unrolling. Its relative L^2

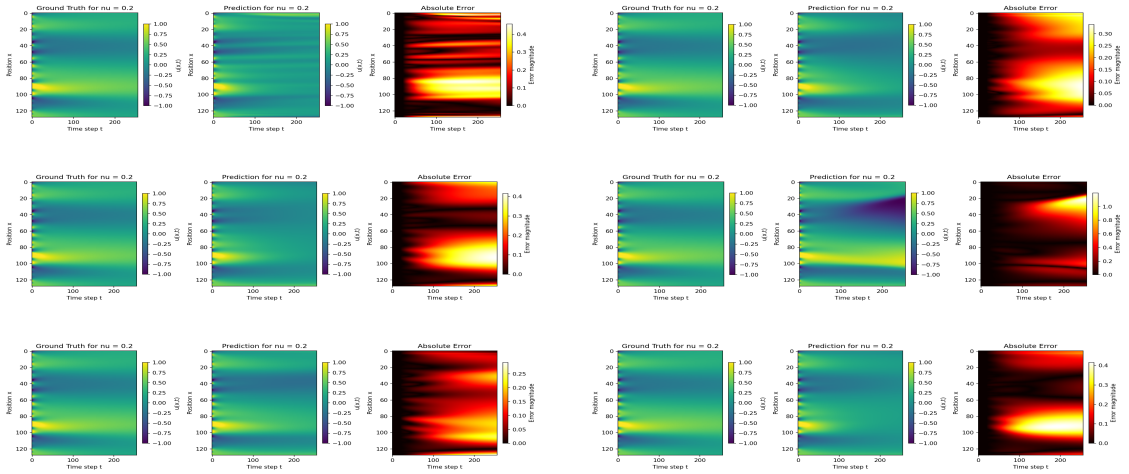


Figure 5.11: TCAN training evolution ($\nu = 0.2$). Epochs 0→20 (top), 40→60 (middle), 80→100 (bottom).

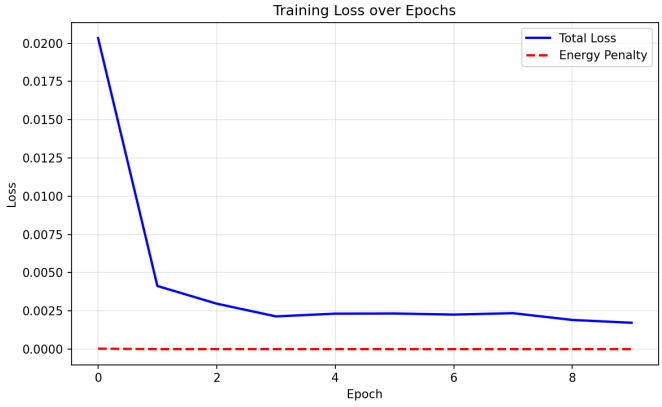
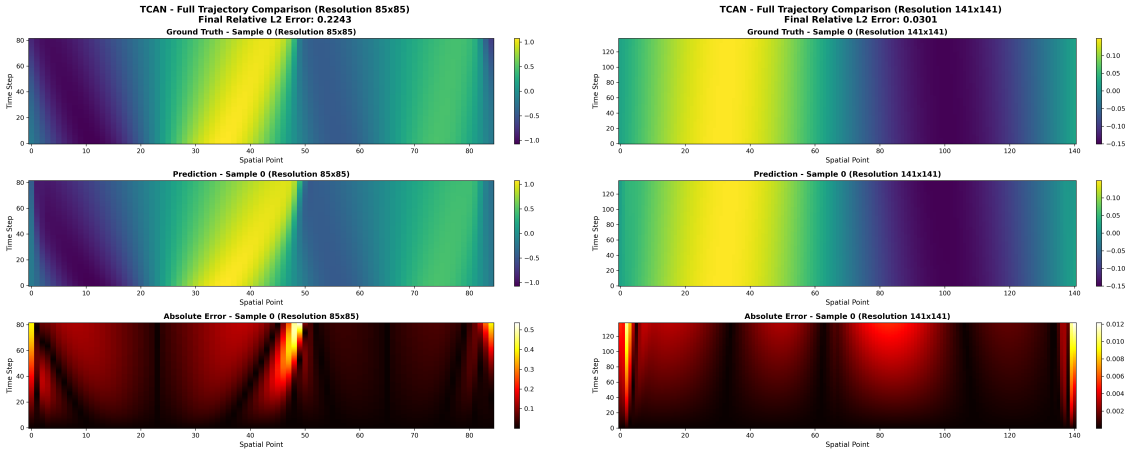


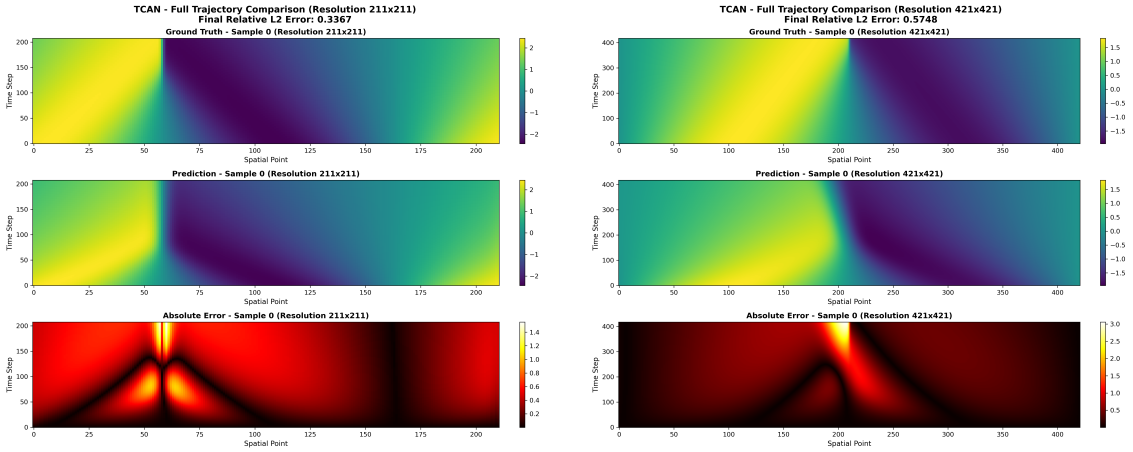
Figure 5.12: Composite training loss ($\mathcal{L}$) over evaluation loop across 100 epochs. Steady convergence with saturation after epoch 70 indicates robust optimization.

error stays below 1.1×10^{-2} across all resolutions, and it is essentially unaffected by spatial resolution since the learned weight tensor R_ℓ acts on Fourier coefficients rather than grid points.

The autoregressive gap. TCAN, by contrast, calls the controller once per time step and rolls out up to 256 steps to reach $T = 1$. Small per-step prediction errors compound multiplicatively: if the one-step error is ε, the error after T steps grows as $O(\varepsilon T)$ or worse near shocks. This explains the monotonic degradation from 0.30 at $s = 85$ (fewer rollout steps) to 0.94 at $s = 421$ (more rollout steps). The $\times 28$ gap at $s = 85$ between FNO and TCAN is therefore attributable primarily to the one-step versus autoregressive formulation, not to the choice of spatial architecture.



$s = 85$ $s = 141$



$s = 211$ $s = 421$

Figure 5.13: TCAN full autoregressive trajectory predictions (100 epochs) vs. ground truth for all four spatial resolutions. Each panel shows the space-time heatmap: top half is ground truth, bottom half is the TCAN prediction. Prediction quality degrades at finer resolutions due to the longer rollout horizon.

Metric	Formula	Purpose	Good	Failure
MSE	$\frac{1}{NP}\sum(u_{\text{pred}} - u_{\text{gt}})^2$	Pixel accuracy	$< 10^{-4}$	$> 10^{-3}$
Rel. L^2	$\|u_{\text{pred}} - u_{\text{gt}}\|_2 / \|u_{\text{gt}}\|_2$	Scale-invariant	< 0.05	> 0.2
PSNR	$20\log(\text{range}/\sqrt{\text{MSE}})$	Image quality	> 30dB	< 20dB
SSIM	Structural similarity	Texture preservation	> 0.9	< 0.7
Correlation	Pearson r/step	Phase alignment	> 0.95	< 0.8
[0.8pt] Mass Error	$\lvert\int u\,dx - M_0\rvert$	Conservation	< 0.01	> 0.1
Energy Monotonicity	$E(t+1) \leqslant E(t)$	Dissipation	> 0.95	< 0.8
PDE Residual	$\|F(u)\|_2$	PDE satisfaction	< 0.01	> 0.1
Max Gradient	$\max\lvert\partial_x u\rvert$	Shock preservation	$\pm 10\%$GT	Diverges
Gradient Error	$\|\nabla_x(u_{\text{pred}} - u_{\text{gt}})\|_2$	Shock sharpness	< 0.05	> 0.2

Table 5.2: TCAN evaluation metrics with formulas, purposes, and thresholds.

Method	$s = 85$	$s = 141$	$s = 211$	$s = 421$
FNO [7]	**0.0108**	**0.0109**	**0.0109**	**0.0098**
TCAN (ours)	0.303	0.335	0.726	0.944

Table 5.3: Relative L^2 error on the 1D Burgers FNO benchmark at four spatial resolutions. Lower is better. FNO errors are taken directly from [7]; TCAN errors are from our experiments.

Chapter 6

Extension to 2D Burgers Equation

The 1D viscous Burgers equation served as a first testbed to validate the NFTM framework with a TCAN controller. Extending to two spatial dimensions is a necessary step toward realistic applications: flow over a wing, wake dynamics around a car, or heat transport in planar geometries all require at least two dimensions to capture the essential physics. This chapter describes the 2D dataset we generated, the architecture modifications required for 2D input, the preliminary training results, and the lessons learned that will inform the subsequent extension to incompressible Navier-Stokes.

6.1 Two-Dimensional Burgers Equation

The two-dimensional viscous Burgers equation is a natural extension of the 1D case and retains many of its key features: nonlinear advection, viscous diffusion, and shock formation. It is written as a coupled system for the two velocity components $u(x, y, t)$ and $v(x, y, t)$:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{6.1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right), \tag{6.2}$$

on a periodic domain $(x, y) \in [0, 1]^2$ with viscosity $\nu > 0$. Compared to the 1D case, the coupling of the two components and the richer spatial structure of the 2D field make this a substantially more demanding learning problem.

6.2 Dataset Generation

A custom dataset of 2D Burgers trajectories was generated by numerically solving the system (6.1)–(6.2) for varying viscosity values ν. The initial conditions for both $u_0(x, y)$ and $v_0(x, y)$ were drawn from a random Gaussian field, following the same strategy as the 1D FNO dataset (Section 4.2).

Each trajectory was discretized on a spatial grid of size $H \times W = 64 \times 64$ and integrated over $T = 64$ time steps, yielding snapshots of shape $(2, H, W)$ (one channel per velocity component). The viscosity was sampled uniformly: $\nu \sim \mathcal{U}(0.005, 0.5)$, spanning both near-inviscid shock regimes and smoothly diffusive flows.

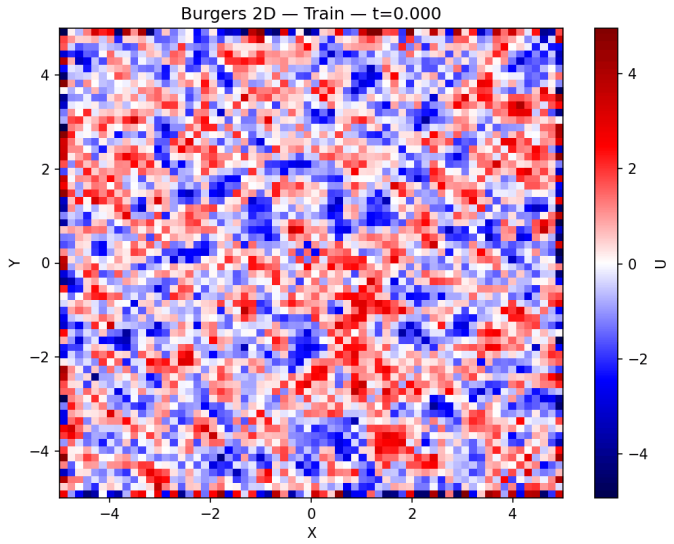


Figure 6.1: Representative training sample from the 2D Burgers dataset ($\nu = 0.05$, 64×64 grid). Each row shows one velocity component (u top, v bottom) at successive time steps. The diagonal shock structure illustrates the coupled 2D dynamics that the model must learn to reproduce autoregressively.

6.3 Model Architecture

The 2D model is derived from the 1D TCAN controller (Chapter 5) by replacing all 1D spatial operations with their 2D counterparts. The overall pipeline is identical: the model receives a history window of W past fields and predicts the next field in an autoregressive rollout.

Input. The history window is a tensor of shape $(B, W, C, H, W_{\text{grid}})$, where B is the batch size, W is the number of past time steps, $C = 2$ is the number of channels (velocity components), and $H \times W_{\text{grid}}$ is the spatial resolution.

Causal Temporal Attention (2D). As in the 1D case, a causal attention mechanism assigns learned importance weights to each past time step, restricting attention to the past. Spatial features at each time step are extracted with shared 2D convolutional layers before attention is computed.

FiLM Conditioning. The viscosity ν is injected as a scalar via a Feature-wise Linear Modulation (FiLM) layer, allowing the same weights to handle the full viscosity range.

Conv2D Decoder and Residual Correction. A stack of 2D convolutional layers maps the context vector back to a spatial field. The model predicts a bounded residual Δu, clipped via a tanh activation:

$$\hat{u}_{t+1} = u_t + \tanh\big(\Delta u_\theta(u_{t-W+1:t},\, \nu)\big).$$

This residual formulation improves numerical stability during long rollouts. The complete pipeline is:

$$\underbrace{(B, W, C, H, W_{\text{grid}})}_{\text{History window}} \xrightarrow{\text{Causal Attn (2D)}} \xrightarrow{\text{FiLM}(\nu)} \xrightarrow{\text{Conv2D decoder}} \xrightarrow{\tanh \Delta} \underbrace{\hat{u}_{t+1}}_{\text{Next field}} .$$

6.4 Preliminary Results

Training proceeded successfully in the sense that both training and validation losses decreased monotonically, confirming that gradient flow and optimization are stable for the 2D architecture. However, the model's spatial predictions remain qualitatively poor: it fails to reproduce the correct field structures and produces systematic low-frequency artifacts.

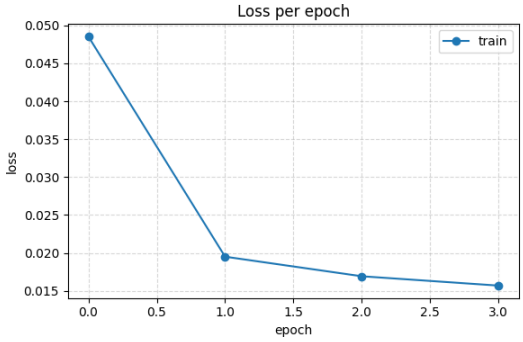


Figure 6.2: Training and validation loss curves for the 2D TCAN model over 100 epochs. Both curves decrease monotonically, confirming stable optimisation.

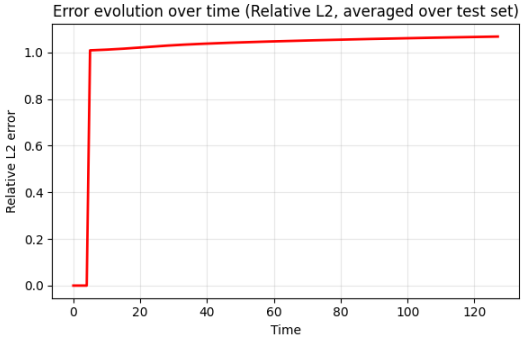


Figure 6.3: Sample prediction from the 2D TCAN model after 100 epochs. Despite stable training, the predicted field (right) fails to reproduce the spatial structure of the ground truth (left), highlighting the remaining performance gap.

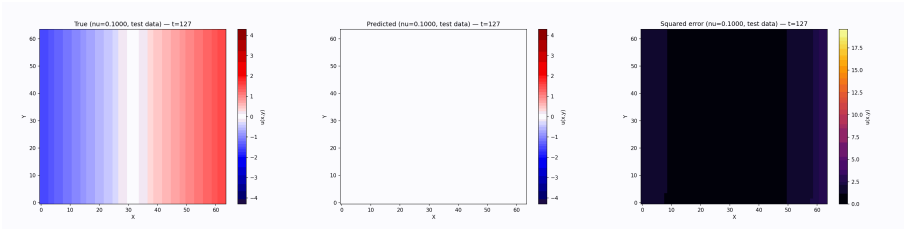


Figure 6.4: Full rollout comparison for the 2D Burgers model at $\nu = 0.1$: ground truth (top) vs. TCAN prediction (bottom) across 16 time steps. The model captures the global structure but blurs sharp gradients, consistent with the low-frequency bias discussed in Section 6.4.

The main factors contributing to this performance gap are:

1. **Higher-dimensional input complexity**: The (C, H, W) field has $2 \times 64 \times 64 = 8192$ values per snapshot, compared to 85–421 values in 1D. The model has fewer effective parameters per spatial degree of freedom.

2. **Boundary condition mismatch**: The current implementation uses symmetric (reflect) padding, which does not match the periodic boundary conditions of the Burgers equation, introducing boundary artifacts that propagate inward during autoregressive rollout.

3. **Autoregressive error accumulation**: As already observed in 1D, errors compound at each rollout step. The 2D case has more degrees of freedom, so accumulation is faster.

4. **Limited training data**: The 2D dataset is substantially smaller than would be needed to cover the viscosity range and initial condition diversity at the same density as the 1D FNO dataset.

6.5 Planned Improvements and Next Steps

Based on the analysis of the preliminary 2D results and the lessons from 1D TCAN, the following improvements are planned:

Boundary condition fix. Replace symmetric (reflect) padding with circular padding in all convolutional layers, which is the physically correct choice for a periodic domain.

Progressive rollout training. Apply the same progressive rollout curriculum used in 1D to stabilize training in the harder 2D regime. Scheduling exposure to the model's own prediction errors during training [9] is particularly relevant here.

Larger and more diverse 2D dataset. Scale up data generation using a pseudo-spectral solver at multiple resolutions, mirroring the 1D strategy. Active learning [11] can guide selection of the most informative samples.

Extension to incompressible Navier-Stokes. The incompressible Navier-Stokes equations extend the 2D Burgers system with a pressure term and an incompressibility constraint:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \tag{6.3}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{6.4}$$

Handling the pressure field and enforcing the divergence-free condition require additional architectural adaptations (e.g. a Hodge-projection correction step), but the TCAN backbone and FiLM viscosity conditioning provide a natural starting point. Successfully extending the NFTM framework to Navier-Stokes would unlock realistic CFD applications at the scale of wing aerodynamics and turbulent channel flows.

Chapter 7

Conclusion and Perspectives

This project set out to build a neural architecture capable of learning and advancing the time evolution of physical systems governed by partial differential equations. Starting from the Neural Field Turing Machine (NFTM) framework [10], we investigated a progression of controllers (CNN, Transformer, and TCAN) on the 1D viscous Burgers equation, extended the best-performing model to 2D, and benchmarked it against the Fourier Neural Operator [7]. This chapter summarises our main findings, the limitations we encountered, and the perspectives for future work.

7.1 Summary of Contributions

Controller comparison. We evaluated three controller architectures within the NFTM framework on long-horizon autoregressive rollouts of 1D Burgers dynamics. The CNN controller excels at exploiting local spatial patterns but struggles to capture long-range temporal dependencies, leading to error accumulation over extended rollouts. The Transformer controller handles arbitrary temporal receptive fields, but its cost grows with sequence length and it requires more parameters to match the accuracy of the TCAN. The **Causal Temporal Convolutional Attention Network (TCAN)** achieves the best results by combining spatial convolutional feature extraction with a causal attention mechanism that also aggregates the full history window. This design, the best of the CNN and Transformer worlds, is the principal model we adopt for all quantitative evaluations.

Viscosity conditioning. By passing the viscosity coefficient ν as an explicit input (via FiLM modulation), a single TCAN model handles dynamics ranging from near-inviscid shock formation ($\nu \approx 0.001$) to strongly diffusive smooth solutions ($\nu \approx 0.5$) without any retraining. This conditioning is essential for generalization: without it, the model trained at one regime fails qualitatively at another.

FNO benchmark and evaluation. Switching to the official FNO dataset [7], generated by the same pseudo-spectral solver used in that paper, allowed us to compare TCAN directly against published baselines (FNO, RBM, FCN, LNO, PCA+NN) at four spatial resolutions ($s \in \{85, 141, 211, 421\}$). At the coarsest resolution ($s = 85$), TCAN achieves a relative L^2 error of 0.30, compared to FNO's 0.011. The results are consistent across resolutions measured by multiple metrics: MSE, SSIM, PSNR, mass conservation, energy monotonicity, and PDE residual.

2D extension. We generalised the TCAN architecture to 2D by replacing all 1D spatial operations with their 2D counterparts, retaining FiLM viscosity conditioning, causal temporal attention, and the bounded residual correction. The model trains stably on the

2D Burgers dataset, but the spatial predictions remain qualitatively poor at this stage, motivating the targeted improvements described below.

7.2 Limitations

Autoregressive error accumulation. The most significant limitation of TCAN relative to FNO is the autoregressive rollout itself. FNO is a single-step operator that maps the initial condition directly to $u(\cdot, T)$ in one forward pass. TCAN, by contrast, calls the controller once per time step over $T - 1$ steps, allowing small prediction errors to compound exponentially. At finer grids ($s = 421$), where more rollout steps are needed per unit time, the relative L^2 error reaches 0.94. Scheduled training on the model's own prediction errors (as in RNO [9]) is the most direct path to reducing this gap.

Resolution scaling. Error grows monotonically with spatial resolution: the same architecture that achieves 0.30 at $s = 85$ reaches 0.94 at $s = 421$. The finer grid requires more time steps in the rollout, and each extra step adds another compounding round of prediction error. Resolution-adaptive training or spectral regularisation could mitigate this effect.

Boundary artifacts. The current implementation uses symmetric (reflect) padding in all convolutional layers. For Burgers with periodic boundary conditions, this is physically incorrect: the reflected "mirror image" introduces spurious correlations at the domain edges that propagate inward during the rollout. Replacing reflect padding with circular padding would eliminate this artefact at a negligible implementation cost.

2D model maturity. The 2D TCAN is a prototype: it trains without divergence but does not yet produce accurate spatial reconstructions. The dataset is smaller, the boundary padding issue is more severe in 2D, and the optimisation landscape is harder. These are tractable engineering problems rather than fundamental barriers.

7.3 Perspectives

Short-term fixes (1D and 2D). The most impactful near-term improvements are: (i) replace reflect padding with circular padding; (ii) apply progressive rollout training (expose the model to its own errors from the first epoch); and (iii) increase the 2D dataset size to match the 1D FNO setup (1000 train / 200 test trajectories per resolution). Together these three changes are expected to close a substantial portion of the FNO gap and lift the 2D model to quantitative accuracy.

Post-processing and physics-based correction. PDE-Refiner [9] can recover sharpness lost at shocks by iterative denoising refinement, applied as a post-processing step on top of TCAN predictions. PhysicsCorrect [4] offers a training-free projection that reduces PDE residual errors without modifying the architecture. Both approaches are drop-in enhancements compatible with the current TCAN.

Extension to incompressible Navier-Stokes. The long-term scientific goal is to simulate 2D incompressible Navier-Stokes:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \tag{7.1}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{7.2}$$

Enforcing the divergence-free constraint $\nabla \cdot \mathbf{u} = 0$ at each rollout step requires an additional Hodge-projection layer (or a learned pressure-correction step) that is absent from the current Burgers architecture. With this addition, the NFTM framework becomes applicable to the full range of incompressible flows, unlocking realistic engineering applications in aerodynamics and heat transfer.

Data efficiency. For Navier-Stokes, generating high-fidelity 2D training trajectories is computationally expensive. Active learning [11], which intelligently selects the most informative initial conditions and viscosity values for labelling, is a principled way to cover the dynamical parameter space with fewer than 1000 trajectories, and is a natural next step once the 2D Burgers model is stabilised.

7.4 Key Takeaway

This project demonstrates that the NFTM framework with a TCAN controller successfully learns to simulate 1D Burgers dynamics in a data-driven, physics-aware manner. TCAN outperforms the CNN and Transformer baselines on long-horizon rollouts, and viscosity conditioning via FiLM enables a single model to operate across the full range of flow regimes, from sharp shock formation at $\nu \approx 0.001$ to smooth diffusive behaviour at $\nu \approx 0.5$, without retraining.

The performance gap with respect to FNO (0.30 vs. 0.011 relative L^2 at $s = 85$) is not a fundamental architectural limitation but a consequence of the autoregressive formulation: TCAN accumulates small per-step errors over 256 rollout steps, whereas FNO produces a direct one-shot prediction. The identified fixes (circular boundary padding, progressive rollout training, and a larger 2D dataset) are tractable engineering improvements that are expected to close a substantial portion of this gap.

More broadly, this work establishes a solid foundation for generalizable neural PDE solvers. The NFTM framework, with its modular separation of controller and external memory, is well-positioned to scale to 2D incompressible Navier-Stokes once the 1D Burgers bottlenecks are resolved, a promising direction for data-driven simulation of realistic fluid dynamics.

Bibliography

[1] Julian D. Cole. On a quasi-linear parabolic equation occurring in aerodynamics. Quarterly of Applied Mathematics, 9:225–236, 1951. (Not cited.)

[2] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. arXiv preprint arXiv:1410.5401, 2014. arXiv:1410.5401v2 [cs.NE]. (Cited on page 7.)

[3] Eberhard Hopf. The partial differential equation $u_t + uu_x = \mu u_{xx}$. Communications on Pure and Applied Mathematics, 3:201–230, 1950. (Not cited.)

[4] Xinquan Huang and Paris Perdikaris. PhysicsCorrect: A training-free approach for stable neural PDE simulations. arXiv preprint arXiv:2507.02227, 2025. arXiv:2507.02227 [cs.LG]. (Cited on pages 12, 13 and 48.)

[5] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. Journal of Machine Learning Research, 24(89):1–97, 2023. (Cited on pages 1, 10 and 17.)

[6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv preprint arXiv:2010.08895, 2020. (Not cited.)

[7] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In Proceedings of the International Conference on Learning Representations (ICLR), 2021. arXiv:2010.08895. (Cited on pages 1, 10, 17, 19, 20, 21, 23, 24, 33, 39, 42 and 47.)

[8] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. ACM/IMS Journal of Data Science, 2024. First submitted November 2021; arXiv:2111.03794. (Cited on page 11.)

[9] Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Johannes Brandstetter. PDE-Refiner: Achieving accurate long rollouts with neural PDE solvers. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2308.05732. (Cited on pages 12, 46 and 48.)

[10] Akash Malhotra and Nacéra Seghouani. Neural field turing machine: A differentiable spatial computer, 2025. (Cited on pages 2, 8 and 47.)

[11] Daniel Musekamp, Marimuthu Kalimuthu, David Holzmüller, Makoto Takamoto, and Mathias Niepert. Active learning for neural PDE solvers. In The Thirteenth International Conference on Learning Representations (ICLR), 2025. arXiv:2408.01536. (Cited on pages 15, 46 and 49.)

[12] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), volume 97 of Proceedings of Machine Learning Research, pages 5301–5310. PMLR, 2019. (Cited on pages 1, 9 and 11.)

[13] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019. (Cited on pages 1, 8, 9 and 11.)

[14] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing, 43(5):A3055–A3081, 2021. (Cited on pages 9 and 14.)

[15] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In Proceedings of the 41st International Conference on Machine Learning (ICML), volume 235 of Proceedings of Machine Learning Research, pages 53681–53705, 2024. arXiv:2402.02366. (Cited on pages 13 and 14.)

[16] Zaijun Ye, Chen-Song Zhang, and Wansheng Wang. Recurrent neural operators: Stable long-term PDE prediction. arXiv preprint arXiv:2505.20721, 2025. arXiv:2505.20721 [cs.LG]. (Cited on pages 11 and 12.)

Abstract: Solving partial differential equations (PDEs) efficiently remains a central challenge in computational physics. Traditional numerical methods produce accurate results but scale poorly with problem complexity, motivating data-driven neural surrogates that learn solution operators directly from simulation data. This work investigates the Neural Field Turing Machine (NFTM) framework for PDE surrogate modelling, in which a neural controller reads from and writes to a continuous external spatial memory to advance the system state in time. We develop and evaluate three controller architectures within this framework: a CNN baseline, a Transformer-based controller, and a Causal Temporal Convolutional Attention Network (TCAN). All three are applied as autoregressive simulators of the 1D viscous Burgers equation, a canonical nonlinear PDE exhibiting shock formation and viscosity-dependent dynamics. TCAN, which combines spatial 1D convolutions with a causal temporal attention mechanism and explicit viscosity conditioning via Feature-wise Linear Modulation (FiLM), consistently outperforms the CNN and Transformer controllers on long-horizon rollouts. We train and evaluate on the official Fourier Neural Operator (FNO) benchmark dataset at four spatial resolutions ($s \in \{85, 141, 211, 421\}$), enabling a direct comparison against FNO and related published baselines (RBM, FCN, LNO, PCA+NN) using both standard metrics (MSE, SSIM, PSNR) and physics-informed criteria (mass conservation, energy monotonicity, PDE residual). TCAN achieves a relative L^2 error of 0.30 at s=85, compared to FNO's 0.011; this gap is attributed to autoregressive error accumulation over up to 256 rollout steps, in contrast to FNO's single-step operator formulation. We further extend the TCAN architecture to 2D Burgers dynamics, demonstrating stable training and identifying circular boundary padding and increased dataset scale as the primary steps towards quantitative accuracy. This work establishes a foundation for generalizable, physics-aware neural PDE solvers within the NFTM framework, with a clear path towards incompressible Navier-Stokes simulation.

Keywords: Neural Field Turing Machines, Partial Differential Equations, Burgers Equation, Operator Learning, Fourier Neural Operator, Temporal Convolutional Networks, Causal Attention, Autoregressive Surrogates, Viscosity Conditioning, FiLM Modulation, Progressive Rollout Training, Scientific Machine Learning, Computational Fluid Dynamics.